

Bachelorthesis  
Attribute Stream-Based Access Control im  
Linux-Kernel

Christian Koch

23. Juli 2026

# Inhaltsverzeichnis

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Einleitung</b>                                               | <b>5</b>  |
| 1.1      | Motivation . . . . .                                            | 5         |
| 1.2      | Wissenschaftliche Fragestellung . . . . .                       | 5         |
| 1.3      | Zielsetzung . . . . .                                           | 5         |
| 1.4      | Aufbau der Arbeit . . . . .                                     | 5         |
| <b>2</b> | <b>Grundlagen</b>                                               | <b>6</b>  |
| 2.1      | Zugriffskontrolle und Reference-Monitor-Konzept . . . . .       | 6         |
| 2.2      | Attribute-Based Access Control . . . . .                        | 7         |
| 2.3      | Attribute Stream-Based Access Control . . . . .                 | 7         |
| 2.4      | Policy-Beschreibungen für ASBAC . . . . .                       | 8         |
| 2.5      | Userspace und Kernelspace in Linux . . . . .                    | 8         |
| 2.6      | Linux Security Modules und LSM-Hooks . . . . .                  | 9         |
| 2.7      | eBPF und eBPF-LSM . . . . .                                     | 9         |
| 2.8      | eBPF-Maps und gepinnte Maps . . . . .                           | 10        |
| 2.9      | eBPF-Verifier, Stack und Programmkomplexität . . . . .          | 11        |
| 2.10     | Aya als Rust-Framework für eBPF . . . . .                       | 12        |
| 2.11     | Dynamische Attribute und Policy-Generationen . . . . .          | 12        |
| 2.12     | Überwachung bestehender Zugriffe und File Descriptors . . . . . | 13        |
| <b>3</b> | <b>Anforderungsanalyse</b>                                      | <b>14</b> |
| 3.1      | Funktionale Anforderungen . . . . .                             | 14        |
| 3.2      | Operative Anforderungen . . . . .                               | 15        |
| 3.3      | Entwicklungsbezogene Anforderungen . . . . .                    | 16        |
| <b>4</b> | <b>Konzeption</b>                                               | <b>17</b> |
| 4.1      | praktische Einschränkungen von eBPF . . . . .                   | 17        |
| 4.2      | Architektur . . . . .                                           | 17        |
| 4.3      | Datenfluss . . . . .                                            | 19        |
| 4.3.1    | Policy-Datenfluss . . . . .                                     | 19        |
| 4.3.2    | Datenfluss dynamischer Attribute . . . . .                      | 19        |
| 4.3.3    | Zugriffsentscheidungen im Kernel . . . . .                      | 19        |
| 4.3.4    | Nachträgliche Kontrolle bestehender Zugriffe . . . . .          | 20        |
| 4.4      | Komponenten . . . . .                                           | 20        |
| 4.4.1    | Userspace-Komponenten . . . . .                                 | 20        |
| 4.4.2    | Kernelspace-Komponenten . . . . .                               | 21        |
| 4.4.3    | Administrationskomponente . . . . .                             | 22        |
| 4.5      | Datenobjekte . . . . .                                          | 22        |
| 4.5.1    | Datenobjekte im Userspace . . . . .                             | 22        |
| 4.5.2    | Datenobjekte im Kernelspace . . . . .                           | 22        |
| 4.6      | Entwurfsentscheidungen . . . . .                                | 23        |
| 4.6.1    | Trennung von Userspace und Kernelspace . . . . .                | 23        |
| 4.6.2    | Einsatz von eBPF-LSM statt klassischem Kernelmodul . . . . .    | 23        |
| 4.6.3    | Auswahl des LSM-Hooks . . . . .                                 | 24        |
| 4.6.4    | Policy-Verarbeitung im Userspace . . . . .                      | 25        |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 4.6.5    | Reduzierte Policy-Repräsentation im Kernelspace . . . . .     | 25        |
| 4.6.6    | Kommunikation über eBPF-Maps . . . . .                        | 26        |
| 4.6.7    | Generationenmodell für Policy-Aktualisierungen . . . . .      | 27        |
| 4.6.8    | Trennung statischer Policies und Streampolicies . . . . .     | 28        |
| 4.6.9    | Aufteilung der Policy-Maps nach Hook und Policy-Typ . . . . . | 29        |
| 4.6.10   | Tail-Call-basierte Aufteilung der eBPF-Programme . . . . .    | 29        |
| 4.6.11   | Combining-Strategie für Policy-Entscheidungen . . . . .       | 30        |
| 4.6.12   | Identifikation von Dateien über Device und Inode . . . . .    | 31        |
| 4.6.13   | Nachträgliche Überwachung durch den Userspace-PEP . . . . .   | 32        |
| 4.6.14   | Enforcement durch Schließen von File Descriptors . . . . .    | 32        |
| 4.6.15   | Administration und Diagnose über ein separates Tool . . . . . | 33        |
| <b>5</b> | <b>Entwurf und Umsetzung</b>                                  | <b>35</b> |
| 5.1      | Implementierungsstruktur . . . . .                            | 35        |
| 5.1.1    | Aufteilung des Rust-Workspaces . . . . .                      | 35        |
| 5.1.2    | Abhängigkeiten zwischen den Crates . . . . .                  | 36        |
| 5.2      | Gemeinsame Datenstrukturen und Map-ABI . . . . .              | 37        |
| 5.2.1    | Kernelgeeignete Policy-Strukturen . . . . .                   | 37        |
| 5.2.2    | Repräsentation dynamischer Attribute . . . . .                | 38        |
| 5.2.3    | Objektidentitäten im Map-Schlüssel . . . . .                  | 38        |
| 5.2.4    | Aufbau und Pinning der eBPF-Maps . . . . .                    | 39        |
| 5.3      | Initialisierung und Laufzeitsteuerung . . . . .               | 40        |
| 5.3.1    | Laden und Prüfen des eBPF-Programms . . . . .                 | 40        |
| 5.3.2    | Initialisierung der Maps und Tail Calls . . . . .             | 40        |
| 5.3.3    | Start der Userspace-Komponenten . . . . .                     | 40        |
| 5.4      | Verarbeitung und Laden von Policies . . . . .                 | 40        |
| 5.4.1    | Einlesen der Policydateien . . . . .                          | 40        |
| 5.4.2    | Syntaktische Verarbeitung . . . . .                           | 40        |
| 5.4.3    | Semantische Validierung . . . . .                             | 40        |
| 5.4.4    | Übersetzung in Map-Einträge . . . . .                         | 40        |
| 5.4.5    | Generationenwechsel und Rollback . . . . .                    | 40        |
| 5.5      | Bereitstellung dynamischer Attribute . . . . .                | 40        |
| 5.5.1    | Aufbau des Attributverzeichnisses . . . . .                   | 41        |
| 5.5.2    | Parsen und Validieren der Attribute . . . . .                 | 41        |
| 5.5.3    | Attributschlüssel und Hashwerte . . . . .                     | 41        |
| 5.5.4    | Aktualisierung der Attributgeneration . . . . .               | 41        |
| 5.5.5    | Bereitstellung der Zeitinformation . . . . .                  | 41        |
| 5.6      | Implementierung des Kernelspace-PEP . . . . .                 | 41        |
| 5.6.1    | Verarbeitung des file_open-Hooks . . . . .                    | 41        |
| 5.6.2    | Tail-Call-Kette . . . . .                                     | 41        |
| 5.6.3    | Auswertung statischer Policies . . . . .                      | 41        |
| 5.6.4    | Auswertung von Streampolicies . . . . .                       | 41        |
| 5.6.5    | Combining und Durchsetzung . . . . .                          | 41        |
| 5.7      | Implementierung des Userspace-PEP . . . . .                   | 41        |
| 5.7.1    | Ermittlung bestehender Dateizugriffe . . . . .                | 41        |
| 5.7.2    | Erneute Policy-Auswertung . . . . .                           | 41        |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| 5.7.3    | Entzug des betroffenen File Descriptors . . . . .  | 41        |
| 5.8      | Implementierung des Administrationstools . . . . . | 41        |
| 5.9      | Fehlerbehandlung und Konsistenzsicherung . . . . . | 41        |
| 5.10     | Build und Deployment auf dem Zielsystem . . . . .  | 41        |
| 5.10.1   | Cross-Compilation und Build-Artefakte . . . . .    | 41        |
| 5.10.2   | Implementierungsbedingte Grenzen . . . . .         | 41        |
| <b>6</b> | <b>Evaluation</b>                                  | <b>42</b> |
| 6.1      | Testumgebung . . . . .                             | 42        |
| 6.2      | Testszenarien . . . . .                            | 42        |
| 6.3      | Ergebnisse . . . . .                               | 42        |
| <b>7</b> | <b>Zusammenfassung und Ausblick</b>                | <b>43</b> |

# 1 Einleitung

## 1.1 Motivation

Die Sicherheit von Linux-Systemen ist ein fundamentales Anliegen in der heutigen IT-Landschaft. Das Linux Security Module Framework stellt Mechanismen für verschiedene Sicherheitsprüfungen mittels Attribute-Based Access Control im Linux-Kernel dar[18]. LSM-Hooks ermöglichen es, sicherheitsrelevante Kerneloperationen abzufangen und auf dieser Grundlage Zugriffsentscheidungen zu treffen. Ein LSM-Hook prüft einen Zugriff jedoch grundsätzlich zu dem Zeitpunkt, an dem die jeweilige Kerneloperation ausgeführt wird. Bei dauerhaft bestehenden Zugriffen wird eine spätere Änderung der Entscheidungsgrundlage dadurch nicht automatisch erneut bewertet.

## 1.2 Wissenschaftliche Fragestellung

Diese Arbeit untersucht, inwieweit Attribute Stream-Based Access Control im Linux-Kernel prototypisch umgesetzt werden kann. Im Mittelpunkt steht dabei die Frage, wie dynamische Attribute aus dem Userspace für Zugriffsentscheidungen im Kernel bereitgestellt und bei Änderungen nachträglich berücksichtigt werden können.

Daraus ergibt sich folgende wissenschaftliche Fragestellung:

Wie kann Attribute Stream-Based Access Control mit dynamischen Attributen im Linux-Kernel prototypisch umgesetzt werden, und welche technischen Einschränkungen ergeben sich dabei?

Zur Beantwortung dieser Fragestellung werden insbesondere die Überführung textueller Policy-Dateien in eine kernelgeeignete Repräsentation, die Bereitstellung dynamischer Attribute für Zugriffsentscheidungen im Kernel sowie die nachträgliche Überwachung bestehender Zugriffssituationen betrachtet.

## 1.3 Zielsetzung

Zielsetzung der Arbeit ist ein Prototyp, anhand dessen eine Zugriffskontrolle mittels Attribute Stream-Based Access Control im Linux-Kernel stattfindet. Insbesondere sollen Stream-Attribute wie Zeitinformationen sowie externe, zur Laufzeit aktualisierte Attribute in die Zugriffsentscheidung einbezogen werden. Ziel ist die Implementierung eines Prototyps, der Policies einliest, in den Kernel überträgt und Zugriffsentscheidungen anhand dieser Policies trifft. Darüber hinaus soll der Prototyp bereits bestehende Zugriffssituationen bei nachträglich geänderten Bedingungen erkennen und einschränken können.

## 1.4 Aufbau der Arbeit

## 2 Grundlagen

Dieses Kapitel beschreibt die fachlichen und technischen Grundlagen, auf denen der in dieser Arbeit entwickelte Prototyp aufbaut. Zunächst werden grundlegende Konzepte der Zugriffskontrolle eingeordnet, insbesondere das Reference-Monitor-Konzept sowie attributbasierte Zugriffskontrollmodelle. Darauf aufbauend wird erläutert, wie Attribute in dynamischen Systemen als veränderliche Entscheidungsgrundlage verwendet werden können und welche Rolle textuelle Policy-Beschreibungen dabei einnehmen.

Für die Umsetzung im Linux-Kernel sind außerdem die Trennung zwischen Userspace und Kernelspace, Linux Security Modules sowie eBPF relevant. Diese Mechanismen bestimmen, an welchen Stellen Zugriffsentscheidungen technisch getroffen werden können und welche Einschränkungen bei der Implementierung im Kernel gelten. Ein besonderer Fokus liegt dabei auf eBPF-Maps als Schnittstelle zwischen Userspace und Kernelspace, gepinnten Maps für dauerhaft zugreifbare Zustände sowie den Vorgaben des eBPF-Verifiers.

Abschließend werden die für den Prototyp spezifischen Grundlagen betrachtet: die Verwendung von Aya als Rust-Framework, die Abbildung dynamischer Attribute und Policy-Generationen sowie die Überwachung bereits bestehender Zugriffe über File Descriptors. Damit schafft das Kapitel die Grundlage für die anschließende Anforderungsanalyse und die spätere Architektur des Systems.

### 2.1 Zugriffskontrolle und Reference-Monitor-Konzept

Zugriffskontrolle beschreibt Mechanismen, mit denen entschieden wird, ob ein Subjekt eine bestimmte Operation auf einer Ressource ausführen darf. Ein Subjekt kann dabei beispielsweise ein Prozess oder ein Benutzer sein, während Ressourcen Dateien, Netzwerkendpunkte oder andere geschützte Objekte des Systems sein können. Eine Zugriffsentscheidung basiert auf einer Menge von Regeln oder Policies, die festlegen, unter welchen Bedingungen eine Operation erlaubt oder verweigert wird. Diese Sichtweise knüpft an grundlegende Begriffe der Informationsschutzmechanismen an, bei denen Akteure, geschützte Objekte und Berechtigungen als zentrale Elemente der Zugriffskontrolle betrachtet werden [12].

Ein grundlegendes Konzept für die Durchsetzung von Zugriffskontrolle ist der Reference Monitor. Der Reference Monitor ist eine abstrakte Sicherheitskomponente, die jeden Zugriff eines Subjekts auf ein geschütztes Objekt überprüft, bevor dieser Zugriff ausgeführt wird. Damit ein Reference Monitor wirksam ist, muss er insbesondere drei Eigenschaften erfüllen: Er muss bei jedem relevanten Zugriff durchlaufen werden, gegen Manipulation geschützt sein und hinreichend klein beziehungsweise nachvollziehbar sein, damit seine Korrektheit überprüft werden kann [1].

Für Betriebssysteme ist dieses Konzept besonders relevant, da Zugriffe auf Dateien, Prozesse oder andere Systemressourcen letztlich über den Kernel vermittelt werden. Wird Zugriffskontrolle erst außerhalb des Kernels umgesetzt, besteht die Gefahr, dass sicherheitsrelevante Operationen nicht vollständig erfasst oder durch alternative Zugriffspfade umgangen werden. Eine Zugriffskontrolle im Kernel kann dagegen an zentralen Stellen ansetzen, an denen Operationen auf geschützte Ressourcen tatsächlich ausgeführt werden. Dies steht im Zusammenhang mit dem Prinzip der vollständigen Vermittlung, nach dem

jeder Zugriff auf ein geschütztes Objekt vor seiner Ausführung geprüft werden muss [12].

Im Kontext dieser Arbeit bildet das Reference-Monitor-Konzept die Grundlage für die Frage, wie Attribute Stream-Based Access Control im Linux-Kernel umgesetzt werden kann. Der Prototyp soll Zugriffe nicht nur anhand statischer Regeln bewerten, sondern dynamische Attribute in die Entscheidung einbeziehen. Dadurch entsteht zusätzlich die Herausforderung, dass eine zunächst erlaubte Zugriffssituation später unzulässig werden kann, wenn sich entscheidungsrelevante Attribute ändern. Das klassische Reference-Monitor-Konzept erklärt daher zunächst die unmittelbare Kontrolle eines Zugriffs, während die spätere Arbeit zusätzlich betrachtet, wie bestehende Zugriffssituationen nachträglich überwacht werden können.

## **2.2 Attribute-Based Access Control**

Attribute-Based Access Control (ABAC) ist ein Zugriffskontrollmodell, bei dem eine Autorisierungsentscheidung nicht allein anhand einer festen Subjektidentität, wie bei ACL- oder identitätsbasierter Zugriffskontrolle, oder anhand einer zugewiesenen Rolle, wie bei Role-Based Access Control (RBAC), getroffen wird. Stattdessen werden Attribute ausgewertet, die sich auf das Subjekt, die Ressource, die angeforderte Operation und gegebenenfalls auf die Umgebung beziehen. Das National Institute of Standards and Technology (NIST) beschreibt ABAC als logische Zugriffskontrollmethode, bei der solche Attribute gegen Policies, Regeln oder Beziehungen geprüft werden, um zu entscheiden, ob eine Operation zulässig ist [4].

Für diese Arbeit ist ABAC relevant, weil dadurch Zugriffsentscheidungen feiner modelliert werden können als bei rein rollenbasierten oder benutzerbasierten Ansätzen. Ein Subjekt kann beispielsweise über Attribute wie Position, Benutzerkennung oder Gruppenzugehörigkeit beschrieben werden. Eine Ressource kann Attribute wie Pfad, Typ oder Schutzstufe besitzen. Zusätzlich können Umwelt- oder Systemattribute, etwa Uhrzeit oder Systemzustand, in die Entscheidung einbezogen werden. Damit entsteht ein flexibles Modell, in dem Policies Bedingungen über mehrere Attributklassen ausdrücken können.

ABAC trennt damit die Beschreibung der Entscheidungsgrundlage von der konkreten technischen Durchsetzung. Die Policy beschreibt, unter welchen Bedingungen ein Zugriff erlaubt oder verweigert werden soll. Die Auswertung dieser Bedingungen und die Durchsetzung der Entscheidung können dagegen von unterschiedlichen Systemkomponenten übernommen werden. Diese Trennung ist für den Prototyp wichtig, weil die Policy-Verarbeitung im Userspace stattfinden kann, während die Zugriffsdurchsetzung später im Kernel betrachtet wird.

## **2.3 Attribute Stream-Based Access Control**

Attribute Stream-Based Access Control (ASBAC) wird in dieser Arbeit als Erweiterung von ABAC verstanden, bei der Attribute nicht nur als statische Werte betrachtet werden. Stattdessen können Attribute zur Laufzeit aktualisiert werden und dadurch einen Strom von Zustandsänderungen bilden. Eine Zugriffsentscheidung hängt dann nicht nur von den Attributwerten zum Zeitpunkt der ersten Prüfung ab, sondern auch davon, ob sich diese Werte während einer bestehenden Zugriffssituation ändern.

Diese Sichtweise steht inhaltlich in der Nähe von Usage Control. Unter Usage Control

werden Zugriffskontrollmodelle verstanden, die eine Nutzung nicht nur zum Zeitpunkt der initialen Autorisierung betrachten, sondern auch Bedingungen während der laufenden Nutzung einbeziehen. Usage Control erweitert klassische Zugriffskontrolle unter anderem um die Veränderbarkeit von Attributen und die Fortdauer von Entscheidungen während einer Nutzung. Gouglidis et al. beschreiben diese Konzepte als *attribute mutability* und *continuity of decision* [3]. Für ASBAC bedeutet dies, dass ein zunächst erlaubter Zugriff später erneut relevant werden kann, wenn sich ein entscheidungsrelevantes Attribut ändert.

Im Kontext dieser Arbeit ist dieser Punkt zentral. Ein Zugriff auf eine Datei kann beim Öffnen zunächst zulässig sein, weil die aktuellen Attribute die Policy erfüllen. Ändert sich später beispielsweise ein Subjektattribut oder ein Systemattribut, kann dieselbe Zugriffssituation nach der aktuellen Policy-Lage unzulässig werden. ASBAC beschreibt damit den fachlichen Bedarf, Attributänderungen nicht nur für zukünftige Zugriffe, sondern auch für bereits bestehende Zugriffssituationen zu berücksichtigen.

## 2.4 Policy-Beschreibungen für ASBAC

Policies beschreiben die Bedingungen, unter denen eine Zugriffsentscheidung getroffen wird. In einem attributbasierten Modell beziehen sich diese Bedingungen auf Attribute von Subjekt, Ressource, Aktion und Umgebung. NIST beschreibt Policies in ABAC als Regeln oder Beziehungen, gegen die Attribute ausgewertet werden [4]. Für ASBAC bleibt diese Grundidee erhalten, sie wird jedoch um dynamisch veränderliche Attribute erweitert.

Eine Policy-Beschreibung muss dafür ausdrücken können, welche Aktion auf welche Ressource bezogen ist und welche Attributbedingungen erfüllt sein müssen. Im Prototyp kann dies beispielsweise bedeuten, dass das Öffnen einer bestimmten Datei nur erlaubt ist, wenn ein Subjektattribut einen bestimmten Wert besitzt. Die Policy-Datei bleibt dabei eine textuelle Beschreibung im Userspace. Für die Auswertung im Kernel muss diese Beschreibung später in eine kernelgeeignete Repräsentation überführt werden.

Etablierte Zugriffskontrollarchitekturen unterscheiden häufig zwischen Komponenten zur Entscheidung und Komponenten zur Durchsetzung. XACML beschreibt beispielsweise eine Architektur mit Policy Decision Point und Policy Enforcement Point sowie eine Policy-Sprache zur Beschreibung attributbasierter Entscheidungen [10]. Der Policy Decision Point (PDP) wertet eine Zugriffsanfrage anhand der geltenden Policies aus und liefert eine Entscheidung. Der Policy Enforcement Point (PEP) fängt die zu kontrollierende Operation ab und setzt die Entscheidung technisch durch. Diese Arbeit übernimmt nicht XACML als Sprache oder vollständiges Architekturmodell, verwendet die Begriffe aber zur Einordnung der Durchsetzungspunkte. Im Prototyp sind Entscheidung und Durchsetzung teilweise eng gekoppelt: Das eBPF-LSM bildet den Kernspace-PEP für neue Dateiöffnungen, während ein zusätzlicher Userspace-PEP bereits bestehende Dateizugriffe nach Änderungen von Policies oder Attributen erneut bewertet und gegebenenfalls beendet.

## 2.5 Userspace und Kernspace in Linux

Linux trennt zwischen Userspace und Kernspace. Anwendungen laufen im Userspace und greifen auf Betriebssystemfunktionen über Systemaufrufe oder andere definierte Schnittstellen zu. Der Kernspace enthält dagegen den Kernel und damit Code, der privile-



giert ausgeführt wird und zentrale Ressourcen wie Prozesse, Speicher, Dateien und Geräte verwaltet. Die Kernel-Dokumentation beschreibt unterschiedliche Ausführungskontexte, darunter Code, der einem Prozess im Kernelspace zugeordnet ist, sowie Prozesse, die im Userspace laufen [11].

Diese Trennung ist für Sicherheitsmechanismen wesentlich. Komplexe Verarbeitung, Dateiparsing und Validierung sind im Userspace leichter zu entwickeln, zu testen und bei Fehlern weniger kritisch für die Stabilität des Gesamtsystems. Code im Kernelspace hat dagegen unmittelbaren Einfluss auf das Betriebssystem. Fehler in Kernelcode können nicht nur eine Anwendung, sondern das gesamte System beeinträchtigen. Daher ist es sinnvoll, umfangreiche Policy-Verarbeitung im Userspace zu belassen und den Kernelspace auf die sicherheitskritische Durchsetzung vorbereiteter Entscheidungen zu beschränken.

Für diese Arbeit ergibt sich daraus eine Architekturgrenze. Der Userspace liest Policies und Attribute ein, validiert sie und übersetzt sie in eine Form, die im Kernel effizient ausgewertet werden kann. Der Kernelspace übernimmt dagegen die Entscheidung an den relevanten Zugriffspunkten. Diese Aufteilung reduziert die Komplexität im Kernel, ohne die Durchsetzung vollständig aus dem sicherheitsrelevanten Ausführungspfad herauszulösen.

## 2.6 Linux Security Modules und LSM-Hooks

Linux Security Modules (LSM) stellen ein Framework bereit, über das Sicherheitsmodule an sicherheitsrelevanten Stellen des Kernels Entscheidungen treffen können. Die Kernel-Dokumentation beschreibt LSM als Schnittstelle für Sicherheitsmodule wie SELinux, Smack, AppArmor oder TOMOYO [18]. Aus technischer Sicht werden dafür Hooks verwendet, die an bestimmten Punkten im Kernel-Ausführungspfad aufgerufen werden.

LSM-Hooks sind für Zugriffskontrolle besonders relevant, weil sie Operationen an Stellen erfassen können, an denen der Kernel ohnehin über die Zulässigkeit einer Operation entscheidet. Wright et al. beschreiben das LSM-Framework als allgemeine Sicherheitsunterstützung im Linux-Kernel, bei der Hooks Zugriff auf Kernelobjekte vermitteln und Rückgabewerte über Erfolg oder Fehler der Operation entscheiden können [20]. Damit eignen sich LSM-Hooks grundsätzlich als technische Grundlage für einen Reference-Monitor-ähnlichen Durchsetzungspunkt.

Für den Prototyp ist insbesondere der Hook `file_open` relevant. Er erlaubt eine Kontrolle beim Öffnen einer Datei und passt damit zur funktionalen Einschränkung dieser Arbeit auf Dateiöffnungen. LSM-Hooks bewerten jedoch jeweils konkrete Kerneloperationen. Ein beim Öffnen erlaubter Zugriff wird nicht automatisch erneut durch denselben Hook geprüft, wenn sich später Attribute ändern. Diese Eigenschaft erklärt, warum zusätzlich eine Überwachung bestehender Zugriffssituationen betrachtet werden muss.

## 2.7 eBPF und eBPF-LSM

eBPF ist ein Mechanismus, mit dem Programme in einer eingeschränkten Ausführungsumgebung im Linux-Kernel ausgeführt werden können. Die Programme werden vor der Ausführung durch den Kernel geprüft und können an verschiedene Ereignisse oder Hook-Typen gebunden werden. Historisch steht BPF für Berkeley Packet Filter. Die heute im Linux-Kernel verwendete erweiterte Form wird meist als eBPF bezeichnet; die Kernel-

Dokumentation verwendet für das moderne Subsystem jedoch häufig weiterhin die Bezeichnung BPF. In dieser Arbeit wird deshalb eBPF verwendet, wenn der moderne Mechanismus für Programme, Maps und Verifier gemeint ist. Der Begriff BPF wird nur dort aufgegriffen, wo er in offiziellen Bezeichnungen wie `bpf()`-Systemaufruf, BPF-Dateisystem oder in der Kernel-Dokumentation selbst verwendet wird. Die Kernel-Dokumentation beschreibt BPF als im Kernel verwendetes Instruktionsformat und betont, dass BPF-Programme auf definierte Hilfsfunktionen und Kernelobjekte wie Maps beschränkt sind [14].

eBPF-LSM verbindet eBPF mit dem LSM-Framework. Die Kernel-Dokumentation beschreibt LSM-BPF-Programme als Möglichkeit, LSM-Hooks zur Laufzeit durch privilegierte Benutzer zu instrumentieren, um systemweite Mandatory-Access-Control- oder Audit-Policies mit eBPF umzusetzen [15]. Damit kann ein eBPF-Programm an einem LSM-Hook ausgeführt werden und dort eine Entscheidung über die angeforderte Operation zurückgeben.

In den Grundlagen wird eBPF noch nicht als endgültige Lösung begründet, sondern als technischer Mechanismus eingeordnet. Für diese Arbeit ist eBPF interessant, weil es eine Ausführung im Kernel ermöglicht, ohne ein klassisches Kernelmodul zu entwickeln. Gleichzeitig ist eBPF durch Verifier, eingeschränkte Sprachelemente, begrenzten Stack, fehlende allgemeine Heap-Allokation und definierte Hilfsfunktionen deutlich stärker beschränkt als allgemeiner Kernelcode. Diese Einschränkungen müssen in der späteren Konzeption gegen die Vorteile abgewogen werden.

## 2.8 eBPF-Maps und gepinnte Maps

eBPF-Maps sind Datenstrukturen im Kernel, die von eBPF-Programmen und Userspace-Anwendungen gemeinsam genutzt werden können. Die Kernel-Dokumentation beschreibt Maps als generischen Speicher für unterschiedliche Datentypen, der unter anderem den Datenaustausch zwischen Kernelspace und Userspace ermöglicht [16]. Über den Systemaufruf `bpf()` können Maps erzeugt sowie Einträge gelesen, aktualisiert oder gelöscht werden [6].

Für Zugriffskontrolle sind Maps relevant, weil eBPF-Programme nicht beliebige komplexe Userspace-Datenstrukturen direkt verwenden können. Policies und Attribute müssen daher in eine feste, kernelgeeignete Struktur gebracht werden. Hash-Maps oder Arrays können beispielsweise verwendet werden, um Policy-Einträge, Attributwerte, Generationen oder Statusinformationen bereitzustellen. Der Kernel kann diese Werte während einer Zugriffskontrolle auslesen, während der Userspace sie bei Policy- oder Attributänderungen aktualisiert.

Gepinnte Maps erweitern diese Idee um einen stabilen Zugriffspfad im BPF-Dateisystem. Pinning erlaubt es, eBPF-Objekte wie Maps über einen Pfad zugänglich zu machen, sodass andere Prozesse sie später wieder öffnen können [2]. Dies ist für den Prototyp wichtig, weil nicht nur der Loader, sondern auch ein Userspace-PEP oder Administrationswerkzeug den aktuellen Zustand lesen können muss. Gepinnte Maps bilden damit eine zentrale Schnittstelle zwischen den getrennten Userspace-Komponenten und dem Kernelzustand.

## 2.9 eBPF-Verifier, Stack und Programmkomplexität

Bevor ein eBPF-Programm im Kernel ausgeführt werden darf, prüft der eBPF-Verifier dessen Sicherheit. Die Kernel-Dokumentation beschreibt die Prüfung als mehrstufigen Prozess, bei dem unter anderem Kontrollfluss, Registerzustände, Stackzugriffe und Speicherzugriffe analysiert werden [17]. Ein Programm wird nur geladen, wenn der Verifier zeigen kann, dass es die Sicherheitsregeln erfüllt.

Diese Prüfung hat direkte Auswirkungen auf die Gestaltung von eBPF-Programmen. Zugriffe auf Map-Werte, Stackbereiche oder Kontextdaten müssen für den Verifier nachvollziehbar sein. Der Verifier verfolgt unter anderem, ob Register initialisiert sind, welche Zeigerarten verwendet werden und ob Speicherzugriffe innerhalb bekannter Grenzen liegen. Unklare oder zu komplexe Zugriffsmuster können dazu führen, dass ein Programm abgelehnt wird, auch wenn die fachliche Logik korrekt erscheint. Darüber hinaus muss der Kontrollfluss eines eBPF-Programms für den Verifier analysierbar bleiben. Schleifen dürfen nicht unbeschränkt sein, Rekursion ist nicht als allgemeines Programmiermittel verfügbar, und Speicherzugriffe müssen durch bekannte Grenzen abgesichert sein. Auch der Zugriff auf Kernelinformationen erfolgt nicht beliebig, sondern über den Hook-Kontext, Maps oder für den jeweiligen Programmtyp zugelassene Hilfsfunktionen.

Zusätzlich bestehen praktische Begrenzungen bei Stack und Programmkomplexität. Die BPF-Dokumentation nennt für Programme ein Stacklimit von 512 Byte und beschreibt interne Grenzen des Verifiers, darunter die Anzahl der während der Analyse betrachteten Instruktionen und Zustände [14]. Die offizielle Aya-Dokumentation beschreibt eBPF als eingeschränkte Laufzeitumgebung und weist darauf hin, dass eBPF-Programme nur 512 Byte Stack besitzen; bei Programmen, die Tail Calls verwenden, kann der verfügbare Stack auf 256 Byte begrenzt sein [13]. Der Stack eignet sich damit nur für kleine lokale Hilfsdaten, etwa Schlüsselstrukturen für Map-Zugriffe oder wenige temporäre Werte. Größere lokale Puffer, dynamisch wachsende Listen oder Zeichenketten können dort nicht sinnvoll abgelegt werden. Ein allgemeiner Heap, wie er in Userspace-Programmen für dynamische Datenstrukturen wie Listen, Strings oder Objekte genutzt wird, steht eBPF-Programmen nicht zur Verfügung. Veränderliche oder größere Zustände müssen daher typischerweise in eBPF-Maps ausgelagert werden. Für Rust-basierte eBPF-Programme ergeben sich daraus weitere Einschränkungen. Die Standardbibliothek kann im eBPF-Programm nicht verwendet werden; stattdessen steht nur `core` zur Verfügung. Funktionen und Traits, die auf `core::fmt` beruhen, etwa `Display` oder `Debug`, sind nicht nutzbar. Da kein Heap verfügbar ist, können auch `alloc` und darauf aufbauende `Collections` nicht eingesetzt werden. Außerdem darf ein eBPF-Programm nicht auf Panics angewiesen sein, weil die eBPF-Laufzeit weder Stack-Unwinding noch eine Abort-Instruktion unterstützt. Ein eBPF-Programm besitzt zudem keine gewöhnliche `main`-Funktion, sondern wird über einen programmspezifischen Einstiegspunkt an einen Hook gebunden. Da eBPF-Programme häufig direkt Kernel-Speicher oder Hook-Kontextdaten lesen, ist außerdem ein relevanter Teil des Codes als `unsafe` zu behandeln [13]. Für die Arbeit bedeutet dies, dass komplexe Policy-Logik möglichst im Userspace vorbereitet werden sollte, während das eBPF-Programm nur eine kompakte, vorher strukturierte Repräsentation auswertet.

## 2.10 Aya als Rust-Framework für eBPF

Aya ist ein Rust-Framework zur Entwicklung von eBPF-Anwendungen. Die offizielle Aya-Dokumentation beschreibt Aya als Möglichkeit, eBPF-Programme mit Rust zu entwickeln und dabei Cargo, Rust-Strukturen sowie gemeinsame Codeanteile zwischen Userspace und eBPF-Programm zu nutzen [13]. Neben Rust-basierten Frameworks existieren etablierte eBPF-Werkzeuge im C- und Skriptsprachenumfeld. libbpf ist eine C-Bibliothek für die Entwicklung und das Laden von BPF- beziehungsweise eBPF-Programmen, während BCC (BPF Compiler Collection) ein Toolkit für BPF-basierte Analyse-, Tracing- und Monitoring-Werkzeuge ist [5, 9]. Aya stellt dazu eine Rust-basierte Alternative dar.

Für den Prototyp ist Aya vor allem deshalb relevant, weil Userspace-Komponenten und eBPF-Programm in derselben Programmiersprache entwickelt werden können. Gemeinsame Datentypen können in einer gemeinsamen Bibliothek definiert werden, wodurch sich die Gefahr verringert, dass Userspace und Kernel-space unterschiedliche Annahmen über die Struktur von Map-Schlüsseln oder Map-Werten treffen. Gerade bei Policies und Attributen ist diese Konsistenz wichtig, weil dieselben binären Strukturen auf beiden Seiten der Map-Schnittstelle interpretiert werden.

Gleichzeitig hebt Aya die Einschränkungen von eBPF nicht auf. Auch mit Rust gelten die Vorgaben des eBPF-Verifiers, die Stackbegrenzung, der Verzicht auf Heap-Allokation im eBPF-Programm und die Beschränkung auf unterstützte Hilfsfunktionen. Aya ist daher als Entwicklungswerkzeug zu verstehen, nicht als fachliches Zugriffskontrollmodell. Die Entscheidung für Aya betrifft die spätere Implementierung, während die fachlichen Anforderungen weiterhin aus ASBAC und der Kernel-Durchsetzung abgeleitet werden.

## 2.11 Dynamische Attribute und Policy-Generationen

Dynamische Attribute sind Attributwerte, die sich zur Laufzeit ändern können. Im Unterschied zu statischen Eigenschaften, etwa einem festen Dateipfad, können dynamische Attribute zeitabhängig oder durch externe Ereignisse beeinflusst sein. Beispiele sind Uhrzeit, Systemzustand oder vom Nutzer frei definierte Subjektattribute. Die Idee veränderlicher Attribute knüpft an Usage-Control-Modelle an, in denen Attributmutabilität als ausdrückliches Merkmal betrachtet wird [3].

Für den Prototyp bedeutet dies, dass Attribute nicht nur beim Start einmalig geladen werden dürfen. Änderungen müssen vom Userspace erkannt, validiert und in die kernel-geeignete Repräsentation übertragen werden. Da eBPF-Programme zur Laufzeit auf Maps zugreifen können, bieten Maps eine geeignete Schnittstelle, um aktuelle Attributwerte für Zugriffsentscheidungen bereitzustellen [16]. Die Attributnamen und Werte müssen dabei im Userspace so verarbeitet werden, dass das eBPF-Programm mit festen Schlüssel- und Wertstrukturen arbeiten kann.

Policy-Generationen beschreiben zusammengehörige gültige Stände von Policies. Dieser Begriff ist für den Prototyp wichtig, weil Policy-Dateien im Userspace geparkt und validiert werden, bevor sie für Zugriffsentscheidungen verwendet werden. Eine neue Generation sollte erst dann aktiv werden, wenn alle dazugehörigen Map-Einträge vollständig erzeugt und konsistent bereitgestellt wurden. Dadurch wird vermieden, dass der Kernel während einer Aktualisierung auf einen unvollständigen oder widersprüchlichen Policy-Zustand zugreift.

## 2.12 Überwachung bestehender Zugriffe und File Descriptors

Ein Dateizugriff in Linux wird häufig über einen File Descriptor repräsentiert. Der Systemaufruf `open()` liefert bei Erfolg einen nichtnegativen File Descriptor zurück, der in nachfolgenden Systemaufrufen wie `read()`, `write()` oder `lseek()` als Referenz auf die geöffnete Datei verwendet wird [7]. Damit entsteht nach dem Öffnen eine bestehende Zugriffssituation, die unabhängig vom ursprünglichen Pfad weiter über den File Descriptor genutzt werden kann.

Für eine Zugriffskontrolle über `file_open` ist das wichtig, weil der LSM-Hook beim Öffnen der Datei greift. Wenn sich danach entscheidungsrelevante Attribute ändern, wird derselbe bereits geöffnete File Descriptor nicht automatisch erneut durch `file_open` bewertet. Eine ASBAC-orientierte Zugriffskontrolle muss deshalb zusätzlich betrachten, wie bestehende File Descriptors überwacht und bei geänderter Entscheidungsgrundlage behandelt werden können.

Linux stellt über `/proc/<pid>/fd` Informationen über die geöffneten File Descriptors eines Prozesses bereit. Die man-pages beschreiben die Einträge in diesem Verzeichnis als symbolische Links auf die geöffneten Dateien oder andere referenzierte Objekte; der Zugriff darauf unterliegt eigenen Berechtigungsprüfungen [8]. Für den Prototyp kann ein Userspace-PEP diese Informationen nutzen, um zu erkennen, welcher Prozess über welchen File Descriptor auf welche Datei zugreift. Wird eine bestehende Zugriffssituation durch neue Attribute oder Policies unzulässig, kann der Userspace-PEP gezielt den betroffenen Zugriff behandeln, ohne notwendigerweise alle Zugriffe desselben Prozesses zu beenden.

## 3 Anforderungsanalyse

In diesem Kapitel werden die Anforderungen an den zu entwickelnden Prototypen beschrieben. Die Anforderungen ergeben sich aus der Zielsetzung, zu untersuchen, wie Attribute Stream-Based Access Control im Linux-Kernel prototypisch umgesetzt werden kann. Sie beschreiben das Verhalten, das der Prototyp im Rahmen der Arbeit zeigen soll.

### 3.1 Funktionale Anforderungen

Funktionale Anforderungen beschreiben, welche fachlichen Funktionen der Prototyp bereitstellen muss. Sie sind im Regelfall direkt testbar, da geprüft werden kann, ob eine Funktion vorhanden ist und das erwartete Verhalten zeigt. Im Kontext dieser Arbeit betreffen sie insbesondere das Einlesen und Verarbeiten von Policies, die Zugriffskontrolle über LSM-Hooks sowie die nachträgliche Überwachung bestehender Zugriffe.

**FA-01 Policy-Einlesen:** Der Prototyp muss ASBAC-Policies aus einer definierten Quelle im Userspace einlesen können.

*Akzeptanzkriterium:* Eine im Policy-Verzeichnis abgelegte Policy wird erkannt und für die weitere Verarbeitung bereitgestellt.

**FA-02 Policy-Verwaltung:** Der Prototyp muss Änderungen an Policies erkennen und aktualisierte Policies in die Zugriffskontrolle übernehmen können.

*Akzeptanzkriterium:* Wird eine Policy geändert, hinzugefügt oder entfernt, wird diese Änderung vom System verarbeitet.

**FA-04 Zugriffskontrolle bei Dateioffnungen:** Der Prototyp muss Dateioffnungen über den Linux-Security-Module-Hook `file_open` abfangen können.

*Akzeptanzkriterium:* Das Öffnen einer durch eine Policy erfassten Datei wird durch den LSM-Hook `file_open` kontrolliert.

**FA-05 Entscheidungsfindung anhand von Policies:** Der Prototyp muss Zugriffsentscheidungen auf Grundlage der geladenen Policies treffen können.

*Akzeptanzkriterium:* Ein Zugriff wird abhängig von einer Policy erlaubt oder verweigert.

**FA-06 Unterstützung dynamischer Attribute:** Der Prototyp muss dynamische Attribute, beispielsweise Zeitinformationen oder externe Zustandsinformationen, in Zugriffsentscheidungen einbeziehen können.

*Akzeptanzkriterium:* Eine Änderung eines dynamischen Attributs kann zu einer veränderten Zugriffsentscheidung führen.

**FA-07 Überwachung und Enforcement bestehender Dateizugriffe:** Der Prototyp muss bereits bestehende Dateizugriffe überwachen und unmittelbar beenden, wenn sie durch entscheidungsrelevante Policy- oder Attributänderungen unzulässig werden. Als bestehender Zugriff gilt im Rahmen des Prototyps ein geöffneter File Descriptor eines laufenden Prozesses auf eine durch eine Policy erfasste Datei. Der Userspace-PEP ermittelt den zugehörigen Prozess und File Descriptor, bewertet den Zugriff

anhand der aktuellen Policy- und Attributlage und schließt den File Descriptor bei einer Verletzung direkt. Eine reine Ausgabe der Verletzung ohne Enforcement ist nicht vorgesehen.

*Akzeptanzkriterium:* Öffnet ein Prozess eine Datei zunächst zulässig und wird dieser Zugriff anschließend durch eine Policy- oder Attributänderung unzulässig, schließt der Userspace-PEP den betroffenen File Descriptor. Andere File Descriptors desselben Prozesses bleiben geöffnet, sofern sie nicht selbst eine Policy verletzen.

**FA-09 Administrationsschnittstelle:** Der Prototyp soll eine einfache Administrationschnittstelle bereitstellen, über die zentrale Informationen zum Zustand des Prototyps lesend eingesehen werden können.

*Akzeptanzkriterium:* Ein Administrator kann zentrale Informationen zum Zustand des Prototyps abrufen, ohne über die Administrationsschnittstelle Policies, Attribute oder Kernelzustand zu verändern.

**FA-10 Gültige Policy-Generationen:** Der Prototyp muss sicherstellen, dass nur erfolgreich erzeugte und validierte Generationen von Policies für Zugriffsentscheidungen verwendet werden. Fehlerhafte, unvollständige oder nicht erfolgreich verarbeitete Policy-Generationen dürfen nicht als aktive Entscheidungsgrundlage gelten.

*Akzeptanzkriterium:* Eine neue Policy-Generation wird erst dann aktiviert, wenn sie vollständig verarbeitet und erfolgreich validiert wurde. Andernfalls bleibt die zuletzt gültige Policy-Generation aktiv.

**FA-11 Eingabevalidierung:** Der Prototyp muss Eingaben aus dem Userspace validieren, bevor diese im Kernel für Zugriffsentscheidungen verwendet werden.

*Akzeptanzkriterium:* Ungültige oder manipulierte Eingaben werden erkannt und nicht als Entscheidungsgrundlage im Kernel verwendet.

**FA-12 beliebige Attributnamen:** In Policies muss es für den Nutzer möglich sein, Attributnamen frei zu vergeben, ohne dass diese vorher im Prototyp fest definiert sein müssen. Zulässige Attributnamen dürfen nicht leer sein und bestehen aus ASCII-Buchstaben (A-Z, a-z), Ziffern (0-9), Unterstrich (\_) und Bindestrich (-).

*Akzeptanzkriterium:* Eine Policy mit einem gültigen, zuvor nicht bekannten Attributnamen, beispielsweise `subject.position == engineer`, wird erfolgreich verarbeitet. Eine Policy mit einem Attributnamen außerhalb des zulässigen Zeichensatzes wird abgelehnt.

## 3.2 Operative Anforderungen

Operative Anforderungen beschreiben, wie gut der Prototyp seine Funktionen im Betrieb erfüllen soll. Sie beziehen sich auf Eigenschaften, die für Nutzung, Betrieb und Evaluation sichtbar sind, beispielsweise Stabilität, Beobachtbarkeit, Nachvollziehbarkeit, Performance und Reproduzierbarkeit.

**OA-01 Stabilität:** Der Prototyp darf die Stabilität des Linux-Kernels nicht unnötig gefährden.

*Akzeptanzkriterium:* Fehlerhafte oder ungültige Eingaben aus dem Userspace führen nicht zu einem Kernel-Absturz.

**OA-02 Beobachtbarkeit und Nachvollziehbarkeit:** Der Prototyp soll zentrale Zustände und Entscheidungen so sichtbar machen, dass Betrieb und Evaluation nachvollziehen können, welche Policies, Attribute oder Zustände eine Zugriffsentscheidung beeinflusst haben.

*Akzeptanzkriterium:* Für zentrale Zugriffsentscheidungen ist erkennbar, welche Policy oder welcher Zustand die Entscheidung beeinflusst hat.

**OA-03 Performance:** Die zusätzlichen Prüfungen durch den Prototyp sollen den kontrollierten Zugriff nicht unverhältnismäßig verlangsamen.

*Akzeptanzkriterium:* Die Auswirkungen auf die Ausführungszeit werden in der Evaluation betrachtet.

**OA-04 Reproduzierbarkeit:** Die Evaluation des Prototyps soll unter dokumentierten Bedingungen wiederholbar sein.

*Akzeptanzkriterium:* Testumgebung, Policies und Testszenarien werden so beschrieben, dass die Ergebnisse nachvollzogen werden können.

### 3.3 Entwicklungsbezogene Anforderungen

Entwicklungsbezogene Anforderungen beschreiben Eigenschaften, die vor allem für die Entwicklung und spätere Erweiterung des Prototyps relevant sind. Sie sind für Nutzer nicht immer unmittelbar sichtbar, beeinflussen aber, wie verständlich, änderbar und erweiterbar das System bleibt.

**EA-01 Wartbarkeit:** Die Implementierung soll modular aufgebaut sein, sodass Userspace-Komponenten, Kernel-Komponenten und Kommunikationsschnittstelle getrennt betrachtet werden können.

**EA-02 Erweiterbarkeit:** Der Prototyp soll so entworfen werden, dass weitere Attribute, Policies oder LSM-Hooks ergänzt werden können.

**EA-03 Begrenzter Kernel-Anteil:** Komplexe Policy-Logik soll möglichst im Userspace verbleiben, um die Komplexität im Kernel zu reduzieren.



## 4 Konzeption

Dieses Kapitel entwickelt die Konzeption des Prototyps auf Grundlage der zuvor formulierten Anforderungen sowie der technischen Randbedingungen von eBPF. Zunächst werden die praktischen Einschränkungen der Ausführung im Kernelspace eingeordnet. Darauf aufbauend werden die Architektur, die Datenflüsse, die Komponenten und die zentralen Datenobjekte des Prototyps beschrieben. Abschließend werden die wesentlichen Entwurfsentscheidungen, ihre Alternativen und ihre jeweiligen Konsequenzen begründet.

### 4.1 praktische Einschränkungen von eBPF

Aus den beschriebenen eBPF-Einschränkungen folgt, dass der Kernelanteil des Prototyps keine frei strukturierte Policy-Sprache auswerten kann. Textuelles Parsen, dynamische Datenstrukturen und komplexe Auswertungslogik würden sowohl den begrenzten Stack als auch die Anforderungen des Verifiers unnötig belasten. Die Policy-Dateien werden deshalb im Userspace gelesen, validiert und in feste, kernelgeeignete Map-Einträge übersetzt.

Für die Policy-Sprache bedeutet dies, dass Bedingungen nur in einer begrenzten, vorher festgelegten Struktur unterstützt werden. Der Kernel verarbeitet keine Zeichenketten dynamisch, sondern vergleicht feste Felder, numerische Werte oder vorberechnete Hashwerte. Attribute werden ebenfalls nicht als frei wachsende Listen im eBPF-Programm gehalten, sondern über geordnete Schlüssel-Wert-Einträge in eBPF-Maps bereitgestellt. Dadurch kann das eBPF-Programm zur Laufzeit kompakte Lookups ausführen, ohne selbst Speicher dynamisch anzufordern. Auch die Rust-spezifischen Einschränkungen wirken sich auf den Entwurf aus. Der Kernelanteil darf keine Standardbibliothek, keine heapbasierten Collections und keine formatierenden Ausgaben über `Display` oder `Debug` voraussetzen. Fehlerfälle müssen daher über explizite Rückgabewerte oder einfache Kontrollpfade behandelt werden, anstatt sich auf Panics zu stützen. Debug-Ausgaben im eBPF-Code müssen auf dafür geeignete, einfache Hilfsmechanismen beschränkt bleiben.

Die Programmlogik muss außerdem so aufgeteilt werden, dass einzelne eBPF-Programme klein genug für Verifier und Stack bleiben. Dies betrifft insbesondere die Auswertung mehrerer Policy-Gruppen. Komplexere Prüfungen werden daher über vorbereitete Map-Strukturen und, soweit erforderlich, über Tail Calls verteilt. Diese Aufteilung reduziert die Programmkomplexität pro eBPF-Programm, führt aber gleichzeitig dazu, dass die Schnittstellen zwischen den Programmteilen und die Stack-Nutzung besonders sorgfältig begrenzt werden müssen.

### 4.2 Architektur

Die Architektur des Prototyps ist in zwei zentrale Bereiche unterteilt: den Userspace und den Kernelspace. Diese Trennung orientiert sich an der grundsätzlichen Struktur von Linux-Systemen und ermöglicht es, komplexe Logik zur Policy-Verarbeitung außerhalb des Kernels auszuführen, während sicherheitskritische Zugriffsentscheidungen direkt im Kernel umgesetzt werden.

- **Userspace:** Im Userspace befinden sich die Komponenten zur Verwaltung und Verarbeitung von Policies. Dieser unterteilt sich in das Hauptprogramm, einen Attribut-loader, einen Userspace-PEP sowie die Policyverwaltung. Das Hauptprogramm lädt

das Kernelprogramm und startet dieses. Der Attributloader lädt Streamingattribute in den Kernelpace. Die Policyverwaltung liest die textuellen Policy-Dateien, parst diese und übersetzt sie in kernelgeeignete Map-Einträge. Der Userspace-PEP überwacht bestehende Dateizugriffe und setzt erkannte Policy-Verletzungen unmittelbar durch das Schließen des betroffenen File Descriptors durch.

- **Kernelspace:** Im Kernelspace wird ein eBPF-Programm an einen LSM-Hook angebunden. Dieser Kernelspace-PEP greift bei neuen Dateioffnungen in die sicherheitsrelevante Operation ein und trifft Zugriffsentscheidungen auf Basis der vom Userspace bereitgestellten Policies. Hierbei wertet das LSM auch Streamingattribute, welche vom Userspace aus in den Kernel gestreamt werden aus.

Die Kommunikation zwischen Userspace und Kernelspace bildet dabei eine zentrale Schnittstelle des Systems. Sie dient dazu, Policies, Attribute, Zeitinformationen und Generationenstände aus dem Userspace für die Kernel-Auswertung bereitzustellen. Zugriffsentscheidungen werden nicht an den Userspace übertragen. Sie verbleiben im jeweiligen Durchsetzungspfad und werden lediglich über dessen Logging sichtbar gemacht. Dadurch kann die Policy-Logik im Userspace verbleiben, während der Kernel die unmittelbare Durchsetzung der Zugriffskontrolle bei Dateizugriffen übernimmt. Für Zugriffe, die bereits durch einen LSM-Hook erlaubt wurden, erfolgt keine automatische erneute Bewertung durch denselben Hook. Diese nachträgliche Überwachung wird durch den Userspace-PEP übernommen.

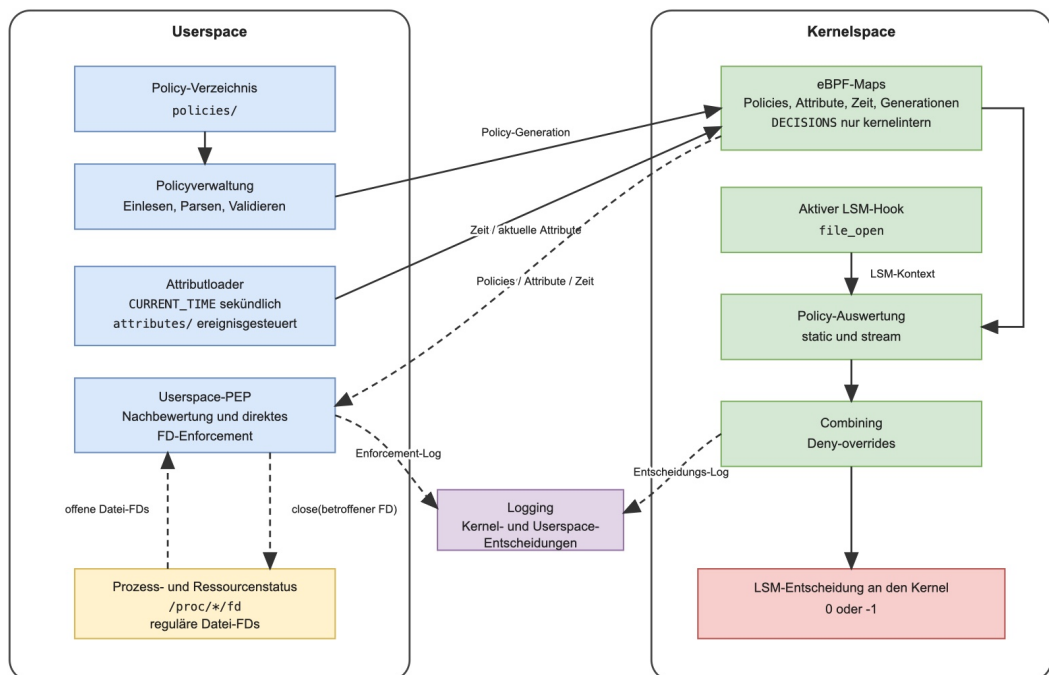


Abbildung 1: Konzeptionelle Architektur des Prototyps

## 4.3 Datenfluss

Der Datenfluss des Prototyps lässt sich in vier Teilbereiche unterteilen. Diese Teilbereiche beschreiben, wie Policies in das System gelangen, wie dynamische Attribute aktualisiert werden, wie ein Zugriff im Kernel bewertet wird und wie bereits bestehende Zugriffe nachträglich überwacht werden.

### 4.3.1 Policy-Datenfluss

Policies werden im Userspace in einem dafür vorgesehenen Policy-Verzeichnis abgelegt. Die Policyverwaltung überwacht dieses Verzeichnis und erkennt, wenn Policies hinzugefügt, geändert oder entfernt werden. Nach einer Änderung werden die vorhandenen Policy-Dateien eingelesen und in kernelgeeignete Map-Einträge übersetzt. Dabei wird geprüft, ob die Policies syntaktisch und semantisch für den Prototyp verarbeitbar sind.

Erst wenn diese Verarbeitung erfolgreich abgeschlossen ist, werden die Policies als neue Policy-Generation in den Kernel übertragen. Die Übertragung erfolgt über gepinnte eBPF-Maps, die als gemeinsame Datenstruktur zwischen Userspace und Kernel space dienen. Durch das Generationenmodell wird verhindert, dass der Kernel während einer Zugriffsentscheidung eine nur teilweise aktualisierte Policy-Menge verwendet. Schlägt die Verarbeitung oder Übertragung fehl, bleibt die zuletzt gültige Policy-Generation aktiv.

### 4.3.2 Datenfluss dynamischer Attribute

Neben den Policies werden dynamische Attribute aus dem Userspace in den Kernel space übertragen. Im Prototyp betrifft dies die aktuelle Zeit sowie vom Nutzer frei wählbare Attribute (vgl. FA-12).

Der Attributloader schreibt die Unix-Zeit einmal pro Sekunde in die einzige gepinnte Zeit-Map `CURRENT_TIME`. Kernel space-PEP und Userspace-PEP lesen denselben Wert und leiten daraus über die gemeinsame Funktion `PolicyTime::from_unix_seconds` die UTC-Komponenten Stunde, Minute und Sekunde ab. Dadurch verwenden beide Ausführungspfade dieselbe Zeitbasis und Konvertierungslogik.

Externe Attribute werden dagegen ereignisgesteuert aktualisiert. Der Attributloader überwacht das Verzeichnis `attributes/` rekursiv. Nach einer erkannten Dateisystemänderung wartet er eine kurze Debounce-Zeit von 100 Millisekunden ab, validiert den neuen Gesamtstand und aktiviert ihn über die Attributgeneration. Ohne Dateisystemereignis findet keine periodische Neuladung dieser Dateien statt.

### 4.3.3 Zugriffsentscheidungen im Kernel

Wenn ein kontrollierter Zugriff stattfindet, wird im Kernel ein LSM-Hook ausgelöst. Bei einem Dateizugriff ist dies beispielsweise der Hook `file_open`. Das eBPF-Programm liest aus dem Hook-Kontext die für die Entscheidung relevanten Informationen aus. Dazu gehören unter anderem das Subjekt, die angefragte Ressource und der aktuelle Prozesskontext.

Auf Grundlage dieser Zugriffsdaten werden die zugehörigen Policies im Kernel ausgewertet. Dabei werden nur Policies berücksichtigt, die für den jeweiligen Hook vorgesehen und auf die konkrete Zugriffssituation anwendbar sind. Eine Policy ist beispielsweise nur

dann anwendbar, wenn Benutzer, Prozessname und Ressource zu der aktuellen Anfrage passen. Erst wenn eine Policy anwendbar ist, kann sie eine Aussage in Form von *Permit* oder *Deny* treffen.

Die Zwischenergebnisse der Policy-Auswertung werden anschließend zusammengeführt. Das Combining entscheidet, ob der Zugriff insgesamt erlaubt oder verweigert wird. Führt das Ergebnis zu einer Verweigerung, gibt das eBPF-LSM-Programm einen Fehlerwert an den Kernel zurück. Der Kernel unterbindet daraufhin den Zugriff.

#### 4.3.4 Nachträgliche Kontrolle bestehender Zugriffe

LSM-Hooks prüfen einen Zugriff zu dem Zeitpunkt, an dem das jeweilige Kernelereignis auftritt. Bei streambasierten Policies kann sich die Entscheidungslage jedoch nachträglich ändern, beispielsweise wenn sich die Uhrzeit oder andere Attribute ändern. Ein bereits geöffneter Zugriff wird dadurch nicht automatisch erneut durch denselben LSM-Hook geprüft.

Aus diesem Grund enthält der Prototyp zusätzlich einen Userspace-PEP. Der Userspace-PEP betrachtet regelmäßig aktive Prozesse und deren aktuell verwendete Ressourcen. Dabei prüft er, ob eine bestehende Zugriffssituation mit den derzeit aktiven Policies vereinbar ist. Wird eine Verletzung erkannt, führt der Userspace-PEP das Enforcement unmittelbar aus und schließt den konkret betroffenen File Descriptor.

Der Userspace-PEP ergänzt damit die kernelbasierte Zugriffskontrolle. Während die LSM-Hooks neue Zugriffe unmittelbar prüfen, adressiert der Userspace-PEP das Problem bereits bestehender Zugriffe bei nachträglich geänderten Bedingungen. Da er die gepinnten Policies und Attribute selbst auswertet und die ermittelte Entscheidung anschließend durchsetzt, enthält diese Komponente neben der PEP-Funktion auch einen Teil der Entscheidungslogik. Eine vollständig getrennte Kommunikation mit einem eigenständigen PDP wird im Prototyp nicht umgesetzt.

### 4.4 Komponenten

Die Architektur des Prototyps setzt sich aus mehreren Komponenten zusammen, die jeweils eine abgegrenzte Aufgabe innerhalb der Zugriffskontrolle übernehmen. Im Folgenden werden die wichtigsten Komponenten beschrieben.

#### 4.4.1 Userspace-Komponenten

Im Userspace befinden sich die Komponenten, die Policies verwalten, dynamische Attribute bereitstellen und bestehende Zugriffssituationen überwachen. Dadurch verbleiben komplexere Verarbeitungsschritte außerhalb des Kernels.

**Policies** Policies beschreiben die Zugriffsvorgaben des Systems. Sie legen fest, welches Subjekt unter welchen Bedingungen auf welche Ressource zugreifen darf oder nicht zugreifen darf. Im Prototyp liegen sie als Textdateien im Policy-Verzeichnis `policies/`. Policies sind damit keine ausführbare Programmlogik, sondern Eingabedaten für die Policyverwaltung und die spätere Auswertung im Kernelspace.

**Policyverwaltung** Die Policyverwaltung bildet die Schnittstelle zwischen den menschenlesbaren Policy-Dateien und den im Kernel verwendbaren Map-Einträgen. Sie überwacht das Verzeichnis `policies/`, liest die dort abgelegten Policies ein, verarbeitet sie und validiert sie vor der Aktivierung. Nach erfolgreicher Validierung übersetzt der Userspace die Policy-Dateien in kernelgeeignete Map-Einträge.

Da Policies während des Betriebs geändert werden können, muss verhindert werden, dass der Kernel eine nur teilweise aktualisierte Policy-Menge verwendet. Dazu aktiviert die Policyverwaltung Policies als atomare Einheit. Dies wird durch ein Generationensystem gewährleistet: Eine neue Policy-Generation wird zunächst vollständig im Userspace vorbereitet und validiert. Erst danach wird sie in den Kernelspace übertragen und als gültige Generation aktiviert. Schlägt dieser Vorgang fehl, bleibt die zuletzt gültige Generation aktiv.

Die Policyverwaltung umfasst damit gelesene Policy-Dokumente, geparste Policies, kernelgeeignete Map-Einträge, Policy-Generationen sowie aktive und zuletzt gültige Policy-Versionen.

**Attributloader** Der Attributloader stellt dynamische Kontextinformationen für die Zugriffskontrolle bereit. Diese Attribute sind nicht notwendigerweise Bestandteil eines einzelnen Zugriffsvorgangs, können aber die Entscheidung beeinflussen. Im Prototyp betrifft dies insbesondere die aktuelle Zeit sowie externe Attribute aus dem Verzeichnis `attributes/`. Die Zeit wird sekundlich in `CURRENT_TIME` geschrieben. Externe Attribute werden nach Dateisystemereignissen im rekursiv überwachten Attributverzeichnis neu validiert und über `ATTRIBUTES` sowie `ATTRIBUTE_GENERATION` aktiviert.

Der Attributloader trifft selbst keine Zugriffsentscheidung. Seine Aufgabe besteht darin, die Entscheidungsgrundlage aktuell zu halten.

**Userspace-PEP** Der Userspace-PEP ergänzt die kernelbasierte Zugriffskontrolle um die nachträgliche Überwachung bereits bestehender Zugriffssituationen. Dies ist notwendig, weil ein LSM-Hook einen Zugriff nur zum Zeitpunkt des jeweiligen Kernelereignisses prüft. Ändern sich danach dynamische Attribute, wird ein bereits bestehender Zugriff nicht automatisch erneut durch denselben Hook geprüft.

Der Userspace-PEP betrachtet deshalb regelmäßig laufende Prozesse und deren verwendete Ressourcen. Dazu nutzt er den Prozess- und Ressourcenstatus des Betriebssystems, beispielsweise Informationen über offene File Descriptors. Diese Informationen werden gegen die aktuell aktiven Policies geprüft. Wird eine Verletzung erkannt, schließt der Userspace-PEP den betroffenen File Descriptor und beendet damit den bereits bestehenden Dateizugriff.

Der Userspace-PEP ersetzt damit nicht die Zugriffskontrolle im Kernelspace, sondern ergänzt sie für Zugriffssituationen, die nach der ursprünglichen Hook-Entscheidung unzulässig werden.

#### 4.4.2 Kernelspace-Komponenten

**Kernelspace-PEP mit eBPF-LSM** Die eBPF-LSM-Komponente bildet den kernelbasierten Durchsetzungspunkt des Prototyps. Sie greift über LSM-Hooks in sicherheitsrelevante Operationen ein und erhält dadurch Zugriff auf die für die Entscheidung relevanten

Zugriffsdaten. Auf dieser Grundlage werden die im Kernelspace vorliegenden Policies ausgewertet. Das Ergebnis dieser Auswertung entscheidet, ob der jeweilige Zugriff erlaubt oder verweigert wird.

#### **4.4.3 Administrationskomponente**

**Administrationstool** Das Administrationstool dient der Diagnose des Prototyps. Es ermöglicht, zentrale Informationen zum Zustand des Systems, über geladene Policies und Attribute lesend einzusehen. Zur Gewährleistung der Konsistenz des Systems wird hier auf eine schreibende Funktion verzichtet.

### **4.5 Datenobjekte**

Neben den aktiven Komponenten besitzt der Prototyp mehrere zentrale Datenobjekte. Diese Datenobjekte beschreiben den Zustand des Systems und bilden die Grundlage für Policy-Verarbeitung, Zugriffsauswertung und Monitoring. Im Folgenden wird zwischen Datenobjekten im Userspace und Datenobjekten im Kernelspace unterschieden.

#### **4.5.1 Datenobjekte im Userspace**

Im Userspace liegen die Datenobjekte, die für die Verwaltung, Vorbereitung und nachträgliche Überwachung von Policies benötigt werden. Dazu gehören zunächst die Policy-Dateien im Verzeichnis `policies/`. Sie stellen die menschenlesbare Beschreibung der Zugriffsvorgaben dar.

Während der Verarbeitung entstehen daraus weitere Datenobjekte: gelesene Policy-Dokumente, geparte Policies und kernelgeeignete Map-Einträge. Diese Zwischenformen sind notwendig, um die textuellen Policies in eine strukturierte Form zu überführen, die später in eBPF-Maps im Kernelspace übertragen werden kann. Zusätzlich verwaltet der Userspace Policy-Generationen. Eine Policy-Generation beschreibt eine zusammengehörige Menge von Policies, die gemeinsam aktiviert oder verworfen wird.

Weitere Datenobjekte im Userspace sind dynamische Attribute wie Zeitinformationen oder externe Zustandswerte aus dem Verzeichnis `attributes/`. Diese Werte werden vom Attributloader aktualisiert und in den Kernelspace übertragen.

Für den Userspace-PEP sind außerdem Prozess- und Ressourceninformationen relevant. Dazu zählen laufende Prozesse, Benutzer-IDs, Prozessnamen, offene File Descriptors und geöffnete Dateien. Diese Informationen stammen aus Betriebssystemquellen wie `/proc/<PID>/fd` und werden verwendet, um bestehende Zugriffssituationen gegen die aktiven Policies zu prüfen.

#### **4.5.2 Datenobjekte im Kernelspace**

Im Kernelspace liegen die Datenobjekte, die für die unmittelbare Zugriffskontrolle an den LSM-Hooks benötigt werden. Dazu gehören die geladenen statischen Policies und Stream-policies. Sie werden nach erfolgreicher Validierung aus dem Userspace übertragen und in eBPF-Maps abgelegt.

Neben den Policies befinden sich auch dynamische Attribute im Kernelspace. Dazu zählen insbesondere die aktuelle Zeit und externe Zustandswerte, die vom Attributloader bereitgestellt werden. Diese Attribute dienen der Auswertung von Streampolicies.

Bei einem Zugriff erzeugt der Kernelspace außerdem Zugriffsdaten. Diese beschreiben die konkrete Anfrage, beispielsweise das Subjekt, den Prozessnamen und die betroffene Dateiressource. Auf Grundlage dieser Zugriffsdaten werden die geladenen Policies ausgewertet.

Schließlich existieren im Kernelspace Zwischenergebnisse der Entscheidungsfindung. Sie halten fest, ob während der Auswertung eine anwendbare Permit- oder Deny-Policy gefunden wurde. Diese Zwischenergebnisse werden im Combining zusammengeführt und bestimmen die finale Entscheidung des LSM-Hooks.

## **4.6 Entwurfsentscheidungen**

### **4.6.1 Trennung von Userspace und Kernelspace**

Das Reference-Monitor-Konzept bildet eine grundlegende theoretische Basis für die Umsetzung von Zugriffskontrollmechanismen. Nach Anderson muss ein Reference Monitor manipulationssicher und verifizierbar sein. Weiterhin muss eine vollständige Überprüfung aller Zugriffe auf geschützte Ressourcen gewährleistet sein [1]. Wright et al. beschreiben, dass das LSM-Framework Sicherheits-Hooks an sicherheitskritischen Stellen des Kernels einfügt und dass diese Hooks Rückgabewerte liefern, welche über Erfolg oder Fehler der angeforderten Operation entscheiden. Daraus folgt, dass die Zugriffskontrollentscheidung innerhalb des Kernel-Ausführungspfades erfolgen muss [20].

Daraus folgt jedoch nicht, dass die gesamte Policy-Verarbeitung im Kernelspace erfolgen muss. Insbesondere das Einlesen, Parsen und Validieren textueller Policy-Dateien ist vergleichsweise komplex und fehleranfällig. Diese Verarbeitungsschritte eignen sich daher besser für den Userspace, da Fehler dort nicht unmittelbar die Stabilität des Kernels gefährden. Der Kernelspace wird im Prototyp auf die sicherheitskritische Auswertung einer bereits vorbereiteten Policy-Repräsentation beschränkt.

Die gewählte Architektur trennt daher zwischen Policy-Verarbeitung im Userspace und Zugriffsdurchsetzung im Kernelspace. Der Userspace übernimmt die Verwaltung der Policies, die Bereitstellung dynamischer Attribute und die Überwachung bestehender Zugriffssituationen. Der Kernelspace übernimmt die unmittelbare Entscheidung an den LSM-Hooks. Damit wird die Komplexität im Kernel reduziert, während die Zugriffskontrolle weiterhin im sicherheitskritischen Ausführungspfad des Kernels durchgesetzt wird.

### **4.6.2 Einsatz von eBPF-LSM statt klassischem Kernelmodul**

Ein klassisches LSM ist Teil des Kernelcodes oder wird als Kernelmodul entwickelt. Änderungen an einem solchen Modul erfordern einen erneuten Build- und Ladeprozess. Für einen experimentellen Prototyp führt dies zu vergleichsweise langen Entwicklungsiterationen.

Gewählt wurde daher die Implementierung als eBPF-LSM. Diese Implementierung erlaubt es, eBPF-Programme an LSM-Hooks auszuführen und damit Zugriffskontrollentscheidungen im Kernel-Ausführungspfad zu treffen [15]. Die Programme können zur

Laufzeit geladen werden und werden vor der Ausführung durch den eBPF-Verifier geprüft [17].

Ein weiterer Grund für diese Entscheidung ist die Möglichkeit, eBPF-Programme mit Rust zu entwickeln. Der Prototyp nutzt dafür Aya. Aya ist eine Rust-Bibliothek für die Entwicklung von eBPF-Programmen und stellt damit eine Alternative zu einer C-basierten Implementierung dar [13]. Dadurch können Userspace- und eBPF-Anteile innerhalb desselben Rust-Workspaces entwickelt werden. Zusätzlich können gemeinsame Datenstrukturen und Teile der Policy-Auswertungslogik in einem gemeinsamen Crate abgebildet werden. Dies verbessert die Konsistenz zwischen Userspace und Kernelspace und reduziert doppelte Implementierungen.

Eine Alternative wäre die Entwicklung eines klassischen LSM gewesen. Diese Alternative bietet grundsätzlich größere Freiheiten im Kernelcode, erhöht jedoch Entwicklungs- und Wartungsaufwand. Für den Prototyp ist eBPF-LSM geeigneter, da Änderungen schneller getestet werden können und gleichzeitig ein sicherheitsrelevanter Ausführungspunkt im Kernel genutzt wird.

Konsequenzen dieser Entscheidung sind, dass die Implementierung die Einschränkungen von eBPF beachten muss, insbesondere die Prüfung durch den Verifier, die begrenzte Stack-Nutzung und die eingeschränkte Programmkomplexität. Diese Einschränkungen beeinflussen unter anderem die Aufteilung der eBPF-Programme in verschiedene Tail-Calls und die reduzierte Policy-Repräsentation im Kernelspace.

#### 4.6.3 Auswahl des LSM-Hooks

Für den Prototyp musste ein LSM-Hook ausgewählt werden, an dem sich die unmittelbare Zugriffskontrolle und die spätere Überwachung bestehender Zugriffssituationen nachvollziehbar untersuchen lassen. Die Wahl fiel auf den Hook `file_open`. Dieser Hook wird beim Öffnen einer Datei ausgelöst und passt damit unmittelbar zu der Anforderung, Dateiöffnungen zu kontrollieren. Gleichzeitig entsteht nach einer erfolgreichen Dateiöffnung ein File Descriptor, über den der Zugriff weiterbestehen kann. Damit eignet sich `file_open` besonders für den Prototyp, weil sowohl die ursprüngliche Entscheidung im Kernel als auch die spätere Überwachung bestehender Dateizugriffe untersucht werden können.

Ein weiterer Grund für diese Auswahl ist die einfache Testbarkeit mit Linux-eigenen Bordmitteln. Dateiöffnungen lassen sich ohne zusätzliche Spezialsoftware mit Werkzeugen wie `cat`, `tail`, Shell-Umleitungen oder kleinen Testdateien auslösen. Andauernde Zugriffssituationen können über geöffnete File Descriptors beobachtet werden. Dafür stellt Linux mit `/proc/<PID>/fd` eine Schnittstelle bereit, über die geöffnete File Descriptors eines Prozesses nachvollzogen werden können [8]. Damit genügt der Hook den Anforderungen des Prototyps, ohne dass eine zusätzliche Testinfrastruktur notwendig wird.

Als Alternative wäre beispielsweise ein Hook wie `socket_bind` möglich gewesen. Ein solcher Hook hätte Netzwerkoperationen betrachtet, etwa das Binden eines Sockets an eine lokale Adresse und einen Port. Für die Zielsetzung dieser Arbeit bringt dies jedoch keinen wesentlichen Vorteil, weil die zentralen Fragestellungen auch anhand von Dateizugriffen untersucht werden können. Zugleich wäre die Test- und Monitoringumgebung aufwendiger, da Netzwerkzustände, Socket-Parameter und laufende Dienste berücksichtigt werden müssten. Aus diesem Grund wird `socket_bind` im Prototyp nicht weiter verfolgt.

Die Konsequenz dieser Entscheidung ist eine bewusste Begrenzung des Prototyps auf



Dateiöffnungen. Der Prototyp trifft damit keine allgemeine Aussage über alle möglichen LSM-Hooks, sondern untersucht ASBAC exemplarisch anhand eines gut beobachtbaren und reproduzierbar testbaren Zugriffstyps.

#### **4.6.4 Policy-Verarbeitung im Userspace**

Policies müssen im Prototyp an einer Stelle verwaltet werden, die für Nutzer nachvollziehbar und für Tests leicht veränderbar ist. Dazu sollen Policies nicht direkt als binäre Map-Einträge geschrieben werden, sondern als textuelle Dateien vorliegen. Diese Dateien können gelesen, ausgetauscht, versioniert und für Tests gezielt verändert werden. Gleichzeitig reicht es nicht aus, die Dateien lediglich unverändert in den Kernel zu übertragen. Sie müssen interpretiert, syntaktisch geprüft, semantisch validiert und in eine kernelgeeignete Repräsentation überführt werden.

Die Entscheidung lautet daher, die Policy-Verarbeitung im Userspace umzusetzen. Eine Userspace-Komponente überwacht das Policy-Verzeichnis, liest die dort abgelegten Policy-Dateien ein, parst sie und validiert sie. Erst wenn diese Verarbeitung erfolgreich abgeschlossen ist, werden daraus strukturierte Einträge für die gepinnten eBPF-Maps erzeugt. Dadurch bleibt die vom Nutzer bearbeitete Policy-Beschreibung von der kompakten Kernelrepräsentation getrennt. Der Nutzer arbeitet mit verständlichen Policy-Dateien, während das eBPF-Programm nur vorbereitete, feste Datenstrukturen auswertet.

Diese Aufteilung ist vor allem durch die technischen Einschränkungen von eBPF begründet. Das Parsen textueller Policies benötigt typischerweise Zeichenkettenverarbeitung, Fehlerbehandlung, Zwischendatenstrukturen und Validierungslogik. Solche Aufgaben sind im eBPF-Programm nur schwer sinnvoll umzusetzen, weil dort kein allgemeiner Heap, keine Standardbibliothek und nur ein stark begrenzter Stack zur Verfügung stehen. Auch Rust-Code im eBPF-Teil kann nicht auf `std`, `alloc` oder normale Collections zurückgreifen. Die komplexe Policy-Verarbeitung im Kernel umzusetzen würde deshalb den eBPF-Code vergrößern, die Verifier-Prüfung erschweren und die Fehlersuche komplizierter machen. Im Userspace können diese Aufgaben dagegen mit normalen Rust-Mitteln umgesetzt und mit aussagekräftigen Fehlermeldungen versehen werden.

Eine Alternative wäre, Policy-Einträge ohne eigene Policyverwaltung direkt in die gepinnten Maps zu schreiben, beispielsweise mit `bpftool`. Diese Variante wäre jedoch fehleranfällig und für Nutzer wenig komfortabel. Die binären Map-Strukturen müssten manuell korrekt erzeugt werden, Validierungsfehler wären schwerer nachvollziehbar, und das Austauschen ganzer Policy-Stände für Tests wäre unübersichtlich. Außerdem bestünde ein höheres Risiko, dass unvollständige oder widersprüchliche Map-Zustände entstehen.

Die Konsequenz dieser Entscheidung ist, dass der Kernelspace keine vollständige Policy-Sprache verarbeitet. Er erhält ausschließlich vorbereitete Map-Einträge und wertet diese im Zugriffspfad aus. Der Userspace übernimmt dagegen die Verwaltung, das Parsen, die Validierung und die Aktualisierung der Policy-Generationen. Damit wird die sicherheitskritische Durchsetzung im Kernel gehalten, während die komplexe und nutzernehe Policyverwaltung außerhalb des Kernels erfolgt.

#### **4.6.5 Reduzierte Policy-Repräsentation im Kernelspace**

Nachdem die Policy-Dateien im Userspace verarbeitet wurden, muss der Kernel die resultierenden Regeln im Zugriffspfad möglichst einfach auswerten können. Der LSM-Hook

wird bei einer sicherheitsrelevanten Operation ausgeführt und liegt damit an einer Stelle, an der unnötig komplexe Programmlogik vermieden werden sollte. Aus Sicht des Reference-Monitor-Konzepts ist außerdem wichtig, dass die durchsetzende Komponente hinreichend klein und nachvollziehbar bleibt. Sie soll jeden relevanten Zugriff prüfen, darf aber selbst nicht zu einer schwer überprüfbaren Policy-Engine im Kernel anwachsen.

Die Entscheidung lautet daher, im Kernel keine vollständige Policy-Sprache abzubilden. Stattdessen werden Policies im Userspace in eine reduzierte, feste Repräsentation übersetzt. Diese Repräsentation besteht aus strukturierten Map-Einträgen mit festen Feldern, etwa Subjekt, Prozessname, Dateidentität, Entscheidung, Stream-Bedingung und einer begrenzten Anzahl dynamischer Attributbedingungen. Das eBPF-Programm muss dadurch nur noch die aktuelle Anfrage mit diesen vorbereiteten Feldern vergleichen und bei zutreffenden Bedingungen eine *Permit*- oder *Deny*-Aussage ableiten.

Diese Reduktion ist durch die Einschränkungen von eBPF notwendig. Komplexe Kontrollflüsse, dynamische Datenstrukturen, umfangreiche Zeichenkettenverarbeitung oder eine frei interpretierbare Policy-Sprache würden den Kernelteil deutlich vergrößern und die Prüfung durch den eBPF-Verifier erschweren. Zusätzlich stehen im eBPF-Programm weder ein allgemeiner Heap noch die Rust-Standardbibliothek zur Verfügung. Die reduzierte Repräsentation begrenzt deshalb den eBPF-Code auf einfache Vergleiche, Map-Zugriffe und klar begrenzte Schleifen. Damit unterstützt sie zugleich die Nachvollziehbarkeit der Durchsetzungskomponente, wie sie aus dem Reference-Monitor-Konzept abgeleitet wird.

Eine Alternative wäre gewesen, die Policy-Funktionalität noch stärker zu reduzieren, beispielsweise nur statische Datei-Policies ohne dynamische Attribute oder nur einzelne fest eingebaute Bedingungen zu unterstützen. Diese Variante hätte den Kernelteil weiter vereinfacht, würde aber den fachlichen Anspruch der Arbeit verfehlen. Gerade die Einbeziehung dynamischer Attribute und die nachträgliche Veränderbarkeit der Entscheidungslage sind für ASBAC wesentlich. Eine andere Alternative wäre eine mächtigere Policy-Auswertung direkt im Kernel. Diese würde zwar mehr Ausdruckstärke bieten, wäre aber mit den eBPF-Beschränkungen und dem Ziel einer kleinen, überprüfbaren Durchsetzungskomponente schlechter vereinbar.

Die Konsequenz ist ein bewusst begrenztes Policy-Modell im Kernel. Nicht jede denkbare Policy-Form kann direkt abgebildet werden. Dafür bleibt die Auswertung im LSM-Hook kompakt, vorhersehbar und auf die Anforderungen des Prototyps zugeschnitten. Erweiterungen müssen daher jeweils prüfen, ob sie in diese feste Kernelrepräsentation passen oder ob zusätzliche Vorverarbeitung im Userspace erforderlich ist.

#### **4.6.6 Kommunikation über eBPF-Maps**

Der Prototyp benötigt eine gemeinsame Datenbasis zwischen Userspace und Kernelspace. Der Userspace muss Policies, Attribute, Zeitinformationen und Generationenstände bereitstellen können. Das eBPF-Programm muss diese Daten im LSM-Hook lesen können, ohne direkt auf Userspace-Speicher oder komplexe Userspace-Datenstrukturen zuzugreifen. Zusätzlich müssen weitere Userspace-Komponenten, insbesondere Userspace-PEP und Administrationswerkzeug, den aktuellen Zustand lesen können.

Die Entscheidung lautet daher, die Kommunikation über eBPF-Maps umzusetzen. Policy-Einträge werden in Array-Maps abgelegt, dynamische Attribute in der Hash-Map `ATTRIBUTES` und ihre aktive Bank in `ATTRIBUTE_GENERATION`. Die gemeinsame Zeit-

basis liegt als Unix-Zeit in der einzigen Zeit-Map `CURRENT_TIME`. Der gemeinsame Typ `PolicyTime` leitet daraus für `Kernelspace-PEP` und `Userspace-PEP` identisch die benötigten UTC-Komponenten ab. Für Zustände, die über mehrere Komponenten hinweg zugreifbar sein müssen, werden gepinnte Maps verwendet. Dadurch erhalten Loader, `Userspace-PEP` und Administrationswerkzeug einen stabilen Zugriffspfad auf dieselben Kernelobjekte, während das `eBPF`-Programm die Werte direkt im Kernel auswerten kann. Die Map `DECISIONS` ist davon ausgenommen: Sie ist nicht gepinnt und speichert nur Zwischenergebnisse zwischen den Tail-Call-Stufen des `Kernelspace-PEP`. Diese Entscheidungen werden nicht in den `Userspace` übertragen, sondern erscheinen ausschließlich in den jeweiligen Logs.

Diese Entscheidung folgt aus dem `eBPF`-Ausführungsmodell. `eBPF`-Programme können nicht beliebige Speicherbereiche des `Userspace` lesen und sollen im Zugriffspfad mit festen, vom Verifier nachvollziehbaren Datenstrukturen arbeiten. `eBPF`-Maps sind der dafür vorgesehene Mechanismus, um Daten zwischen `Userspace` und `eBPF`-Programmen auszutauschen. Sie ermöglichen es, Policy- und Attributzustände im Kernel bereitzustellen und dennoch aus dem `Userspace` heraus zu aktualisieren. Zugleich passen Maps zu der reduzierten Policy-Repräsentation, weil sie feste Schlüssel- und Werttypen erzwingen.

Eine gleichwertige Alternative gibt es im Rahmen dieses Prototyps nicht. Direkte Übergeben über Funktionsparameter sind für dauerhafte Policy- und Attributzustände nicht geeignet, weil der LSM-Hook bei jedem Zugriff unabhängig ausgeführt wird. Ein erneutes Laden des `eBPF`-Programms bei jeder Policy- oder Attributänderung wäre unpraktisch und würde den laufenden Betrieb unnötig stören. `Userspace`-Dateien können vom `eBPF`-Programm ebenfalls nicht als normale Dateien gelesen und interpretiert werden. Damit bleiben `eBPF`-Maps die naheliegende und technisch vorgesehene Schnittstelle.

Die Konsequenz ist, dass alle zwischen `Userspace` und Kernel geteilten Zustände in feste Map-Layouts übersetzt werden müssen. Änderungen an diesen Layouts betreffen beide Seiten und müssen deshalb kontrolliert erfolgen. Gleichzeitig entsteht dadurch eine klare Schnittstelle: Der `Userspace` schreibt validierte Zustände in Maps, der Kernel liest diese Zustände bei der Zugriffskontrolle, und externe Komponenten können über gepinnte Maps lesend auf denselben Zustand zugreifen.

#### **4.6.7 Generationenmodell für Policy-Aktualisierungen**

Policies können während des Betriebs geändert, hinzugefügt oder entfernt werden. Dabei darf der Kernel während einer Zugriffentscheidung nicht auf einen teilweise aktualisierten Policy-Zustand zugreifen. Problematisch wäre beispielsweise, wenn einzelne Map-Einträge bereits überschrieben wurden, andere Einträge derselben Policy-Menge aber noch zum vorherigen Stand gehören. In diesem Fall könnte eine Zugriffentscheidung auf einer widersprüchlichen Mischung aus alter und neuer Policy-Lage beruhen.

Die Entscheidung lautet daher, Policy-Aktualisierungen über ein Generationenmodell umzusetzen. Die Policy-Maps werden in zwei Bänke aufgeteilt. Eine Bank enthält die aktuell aktive Policy-Generation, während die andere Bank als inaktive Bank für die nächste Generation vorbereitet wird. Bei einer Änderung schreibt der `Userspace` zunächst alle vorbereiteten Policy-Einträge in die inaktive Bank. Erst wenn dieser Schreibvorgang erfolgreich abgeschlossen ist, wird der Generationenzähler aktualisiert. Das `eBPF`-Programm liest den aktiven Generationenwert und bestimmt daraus, welche Bank bei der Auswertung

verwendet wird.

Dieses Vorgehen reduziert das Risiko inkonsistenter Zustände. Solange die neue Bank noch nicht vollständig geschrieben ist, bleibt die bisherige Generation aktiv. Schlägt das Parsen, Validieren oder Schreiben der neuen Policies fehl, wird der Generationenzähler nicht umgeschaltet und die letzte gültige Generation bleibt weiter wirksam. Damit passt das Modell zu der Anforderung, nur erfolgreich verarbeitete Policy-Stände als Entscheidungsgrundlage zu verwenden. Dasselbe Grundprinzip wird auch für dynamische Attribute verwendet: Attribute werden bankweise geschrieben und anschließend über eine eigene Attributgeneration aktiviert.

Eine Alternative wäre, Policy-Einträge direkt an ihrer aktiven Position zu überschreiben. Diese Variante wäre einfacher, könnte aber während der Aktualisierung zu Zwischenzuständen führen, die weder der alten noch der neuen Policy-Menge vollständig entsprechen. Eine weitere Alternative wäre, die Zugriffskontrolle während einer Aktualisierung anzuhalten oder das eBPF-Programm vollständig neu zu laden. Beides wäre für einen laufenden Prototyp unnötig invasiv und würde den Betrieb stärker beeinflussen.

Die Konsequenz des Generationenmodells ist ein zusätzlicher Verwaltungsaufwand. Die Maps müssen Platz für mehrere Bänke bereitstellen, und Userspace sowie Kernelspace müssen denselben Mechanismus zur Berechnung der aktiven Bank verwenden. Dafür erhält der Prototyp eine einfache Umschaltlogik, bei der der Kernel entweder die alte oder die neue Generation auswertet, aber keine bewusst gemischte Policy-Menge.

#### **4.6.8 Trennung statischer Policies und Streampolicies**

Im Prototyp existieren Policies, deren Entscheidung allein von den Eigenschaften der aktuellen Anfrage abhängt, und Policies, die zusätzlich dynamische Zustände berücksichtigen. Zu den statischen Kriterien gehören beispielsweise das Subjekt, der Prozessname und die Dateiidentität. Streampolicies verwenden darüber hinaus Werte, die sich zur Laufzeit ändern können, etwa Zeitinformationen oder dynamische Attribute aus dem Attributstrom. Diese beiden Fälle unterscheiden sich in der Auswertung deutlich: Statische Policies benötigen keine zusätzlichen Attribut- oder Zeit-Maps, während Streampolicies solche Zustände lesen und auswerten müssen.

Die Entscheidung lautet daher, statische Policies und Streampolicies getrennt zu repräsentieren und getrennt auszuwerten. Policies ohne Stream-Bedingungen werden in die statische Policy-Map geschrieben. Policies mit Zeit- oder Attributbedingungen werden in die Stream-Policy-Map geschrieben. Im eBPF-Programm werden zuerst die statischen Policies ausgewertet und anschließend die Streampolicies. Die Ergebnisse beider Auswertungsschritte werden danach zusammengeführt.

Diese Trennung reduziert die Komplexität der einzelnen Auswertungspfade. Der statische Pfad bleibt auf einfache Vergleiche der aktuellen Anfrage gegen vorbereitete Policy-Felder beschränkt. Der Stream-Pfad enthält dagegen die zusätzliche Logik für aktuelle Zeitwerte, Attributgenerationen und Attribut-Map-Lookups. Damit muss nicht jede Policy-Auswertung die vollständige dynamische Logik enthalten. Das unterstützt die Begrenzung der Programmkomplexität und erleichtert die Nachvollziehbarkeit der jeweiligen Entscheidung.

Eine Alternative wäre eine gemeinsame Policy-Map mit einem Feld gewesen, das statische und dynamische Policies unterscheidet. Dann müsste ein einzelner Auswertungspfad

jedoch beide Fälle behandeln und bei jeder Policy entscheiden, welche Zusatzlogik relevant ist. Dies würde den eBPF-Code größer machen und die Trennung der Verantwortlichkeiten verwischen. Eine weitere Alternative wäre, ausschließlich Streampolicies zu verwenden und statische Policies als Spezialfall ohne dynamische Bedingung abzubilden. Auch dies würde einfache statische Regeln unnötig durch den komplexeren Stream-Pfad führen.

Die Konsequenz ist ein zusätzlicher Verwaltungsaufwand, weil mehrere Policy-Maps gepflegt werden müssen. Dafür bleibt die statische Auswertung einfach, während dynamische Bedingungen in einem eigenen Pfad behandelt werden. Für den Prototyp ist diese Aufteilung angemessen, weil sie die fachliche Unterscheidung zwischen statischen Regeln und streamabhängigen Regeln auch technisch sichtbar macht.

#### **4.6.9 Aufteilung der Policy-Maps nach Hook und Policy-Typ**

Die Policy-Repräsentation hängt nicht nur davon ab, ob eine Policy statisch oder streamabhängig ist. Sie hängt auch vom kontrollierten Hook ab, weil unterschiedliche LSM-Hooks unterschiedliche Zugriffsdaten bereitstellen. Bei `file_open` stehen beispielsweise Dateiinformationen wie Device und Inode im Mittelpunkt. Andere Hooks würden andere Ressourcen- und Kontextdaten benötigen. Eine gemeinsame Map für alle denkbaren Hook-Typen müsste daher entweder sehr große Einträge enthalten oder viele Felder besitzen, die für den jeweiligen Hook nicht verwendet werden.

Die Entscheidung lautet daher, Policy-Maps nach Hook und Policy-Typ aufzuteilen. Im aktuellen Prototyp ist nur `file_open` aktiv umgesetzt. Dementsprechend existieren getrennte Maps für statische `file_open`-Policies und `file_open`-Streampolicies. Die Struktur ist dennoch so gewählt, dass weitere Hook-Typen grundsätzlich eigene Map-Strukturen erhalten könnten, ohne die bestehende `file_open`-Auswertung aufzublähen.

Diese Aufteilung hält die Map-Einträge klein und an den jeweiligen Zugriffstyp angepasst. Das eBPF-Programm muss im `file_open`-Pfad nur die Daten lesen, die für Dateiöffnungen relevant sind. Dadurch sinkt die Menge an ungenutzten Feldern und die Auswertungslogik bleibt auf den konkreten Hook zugeschnitten. Gleichzeitig passt die Aufteilung zu der Entscheidung, statische Policies und Streampolicies getrennt zu behandeln.

Eine Alternative wäre eine universelle Policy-Map für alle Hooks und Policy-Typen. Diese Variante hätte eine zentrale Datenstruktur, würde aber größere Einträge und zusätzliche Fallunterscheidungen im Kernel erfordern. Eine andere Alternative wäre, alle Policies pro Hook in einer Map zu speichern und nur den Policy-Typ innerhalb des Eintrags zu unterscheiden. Auch dies würde den Auswertungspfad stärker verzweigen.

Die Konsequenz ist, dass neue Hooks nicht nur neue Auswertungslogik, sondern auch eigene Map-Layouts benötigen können. Dieser zusätzliche Aufwand ist bewusst akzeptiert, weil er die aktuell aktive `file_open`-Auswertung einfach hält und Erweiterungen klar von bestehenden Pfaden trennt.

#### **4.6.10 Tail-Call-basierte Aufteilung der eBPF-Programme**

Die Policy-Auswertung besteht aus mehreren Teilschritten. Zunächst muss der Hook-Einstieg die Anfrage erfassen. Danach werden statische Policies ausgewertet, anschließend Streampolicies, und zum Schluss werden die Zwischenergebnisse zu einer finalen

Entscheidung kombiniert. Würden alle Schritte in einem einzigen eBPF-Programm umgesetzt, würde dieses Programm größer und für den Verifier schwerer analysierbar. Gerade bei eBPF sind Programmgröße, Kontrollfluss und Stacknutzung praktische Grenzen, die den Entwurf unmittelbar beeinflussen.

Die Entscheidung lautet daher, die Auswertung über Tail Calls in mehrere eBPF-Programme aufzuteilen. Der Einstieg am LSM-Hook springt über eine Program-Array-Map in den statischen Auswertungsschritt. Dieser springt anschließend in den Stream-Auswertungsschritt, und dieser wiederum in das Combining-Programm. Jeder Schritt übernimmt damit eine klar begrenzte Aufgabe.

Diese Aufteilung reduziert die Komplexität der einzelnen eBPF-Programme. Der statische Pfad muss keine dynamischen Attribute auswerten, der Stream-Pfad muss nicht die gesamte Hook-Initialisierung enthalten, und das Combining kann auf bereits gespeicherte Zwischenergebnisse zugreifen. Gleichzeitig entspricht die Aufteilung der fachlichen Struktur der Entscheidung: Anfrage erfassen, Policy-Gruppen auswerten, Ergebnisse zusammenführen. Sie erleichtert außerdem die spätere Anpassung einzelner Schritte, ohne die gesamte Auswertung in einem großen Programm ändern zu müssen.

Eine Alternative wäre ein monolithisches eBPF-Programm. Diese Variante hätte weniger Tail-Call-Verwaltung benötigt, würde aber die Verifier-Komplexität erhöhen und frühere Probleme mit zu großen Programmen begünstigen. Eine weitere Alternative wäre, weniger Policy-Funktionalität im Kernel abzubilden, um ein einzelnes Programm klein genug zu halten. Das würde jedoch den Anspruch des Prototyps einschränken, statische und dynamische Bedingungen gemeinsam untersuchen zu können.

Die Konsequenz ist, dass eine zusätzliche Tail-Call-Tabelle gepflegt werden muss. Außerdem muss beachtet werden, dass Tail Calls eigene eBPF-Einschränkungen mit sich bringen, insbesondere hinsichtlich Stacknutzung. Dafür bleibt jeder Auswertungsschritt kleiner und besser kontrollierbar.

#### **4.6.11 Combining-Strategie für Policy-Entscheidungen**

Mehrere Policies können auf dieselbe Zugriffssituation anwendbar sein. Dabei können einzelne Policies zu unterschiedlichen Aussagen kommen, etwa *Permit* oder *Deny*. Der Prototyp benötigt daher eine Strategie, mit der die Zwischenergebnisse der Policy-Auswertung zu einer finalen Hook-Entscheidung zusammengeführt werden.

Die Entscheidung lautet, Zwischenergebnisse zunächst in einem kleinen Entscheidungszustand abzulegen und die finale Entscheidung in einem separaten Combining-Schritt zu treffen. Die statische und die streambasierte Auswertung halten fest, ob während ihres jeweiligen Auswertungspfades eine passende Permit- oder Deny-Policy gefunden wurde. Das Combining-Programm liest diesen Zustand und entscheidet daraus, ob der Hook den Zugriff erlaubt oder verweigert. Im aktuellen Prototyp wird eine Deny-overrides-Strategie verwendet: Sobald eine Deny-Entscheidung vorliegt, wird der Zugriff verweigert.

Diese Aufteilung trennt Policy-Matching und finale Konfliktauflösung. Die einzelnen Auswertungspfade müssen nicht selbst die vollständige globale Entscheidung treffen, sondern liefern nur ihre Zwischenergebnisse. Das macht die Struktur des eBPF-Programms nachvollziehbarer und erlaubt es, die Combining-Logik an einer klar abgegrenzten Stelle zu betrachten.

Eine Alternative wäre ein sofortiger Abbruch beim ersten passenden Ergebnis, beispielsweise als Priority-Deny- oder Priority-Permit-Strategie. Dabei würde ein Auswertungspfad unmittelbar mit einem erlaubenden oder verweigernden Rückgabewert terminieren. Diese Variante wäre für eine feste Strategie einfacher, würde aber die spätere Untersuchung anderer Combining-Algorithmen erschweren. Außerdem würde die Reihenfolge der Policy-Auswertung stärker in die Semantik eingehen.

Die Konsequenz ist, dass ein zusätzlicher Entscheidungszustand zwischen den Tail-Call-Schritten benötigt wird. Dafür bleibt die Konfliktauflösung zentralisiert. Der Prototyp verwendet aktuell Deny-overrides, ohne die Policy-Auswertung selbst fest auf einen frühen Abbruch nach der ersten passenden Policy zu beschränken.

#### **4.6.12 Identifikation von Dateien über Device und Inode**

Policies werden für Nutzer über Dateipfade formuliert. Für die Auswertung im Kernel ist ein Pfadvergleich jedoch ungünstig. Pfade sind Zeichenketten, können unterschiedlich geschrieben werden und sind nicht zwingend eine eindeutige Beschreibung des zugrunde liegenden Dateiobjekts. Beispiele sind symbolische Links, relative Pfade oder mehrere Pfade, die auf dasselbe Dateiobjekt verweisen können. Ein Zeichenkettenvergleich im eBPF-Programm wäre zudem unnötig aufwendig und würde die Policy-Auswertung vergrößern.

Die Entscheidung lautet daher, Dateipfade im Userspace in die Dateiidentität aus Device und Inode zu übersetzen. Beim Verarbeiten der Policy wird der angegebene Pfad über Dateimetadaten aufgelöst. In der Policy-Map werden anschließend Device und Inode gespeichert. Beim `file_open`-Hook liest das eBPF-Programm die entsprechenden Werte aus dem Kernelkontext der geöffneten Datei und vergleicht diese mit den vorbereiteten Policy-Werten.

Diese Repräsentation passt besser zum Kernelkontext. Der Kernel arbeitet beim geöffneten Dateiobjekt ohnehin mit Inode- und Superblock-Informationen. Dadurch muss das eBPF-Programm keinen vollständigen Pfad rekonstruieren und keine langen Zeichenketten vergleichen. Außerdem werden unterschiedliche Pfadschreibweisen reduziert, weil die Entscheidung auf der identifizierten Datei und nicht auf der Schreibweise des Pfades basiert.

Eine Alternative wäre ein Pfadvergleich im Kernel. Dafür müssten Pfade oder Pfadbestandteile in Maps gespeichert und im eBPF-Programm verglichen werden. Diese Variante wäre komplexer, anfälliger für Uneindeutigkeiten und stärker von Pfadnormalisierung abhängig. Eine weitere Alternative wäre ein Hash des Pfades. Auch dieser würde jedoch weiterhin an der Pfadschreibweise hängen und nicht am tatsächlich geöffneten Dateiobjekt.

Die Konsequenz ist, dass Policies beim Laden an den zu diesem Zeitpunkt vorhandenen Dateinhalt beziehungsweise dessen Device-Inode-Identität gebunden werden. Wird eine Datei später ersetzt, kann sich die Inode ändern und die Policy muss entsprechend neu verarbeitet werden. Dafür wird die Auswertung im Kernel deutlich einfacher und näher an der tatsächlichen Dateiidentität durchgeführt.

#### 4.6.13 Nachträgliche Überwachung durch den Userspace-PEP

Der LSM-Hook `file_open` bewertet eine Dateiöffnung zu dem Zeitpunkt, an dem die Datei geöffnet wird. Wenn der Zugriff erlaubt wurde, kann der Prozess die Datei anschließend über den erhaltenen File Descriptor weiter verwenden. Ändern sich danach dynamische Attribute oder Policies, wird dieser bereits bestehende Zugriff nicht automatisch erneut durch denselben Hook geprüft. Für ASBAC ist genau dieser Fall relevant, weil eine zunächst zulässige Zugriffssituation durch spätere Attributänderungen unzulässig werden kann.

Die Entscheidung lautet daher, die unmittelbare Hook-Entscheidung durch einen Userspace-PEP zu ergänzen. Der Userspace-PEP liest regelmäßig Informationen über laufende Prozesse und offene File Descriptors aus dem System. Anschließend bewertet er diese bestehenden Dateizugriffe gegen die aktuell aktiven Policies und Attribute. Dazu verwendet er dieselben vorbereiteten Policy- und Attributzustände, die auch dem Kernel über die gepinnten Maps zur Verfügung stehen. Wird eine Verletzung erkannt, führt der Userspace-PEP das Enforcement direkt aus. Die Bezeichnung als PEP beschreibt seine Durchsetzungsfunktion. Da im Prototyp kein separater PDP-Prozess für die Nachbewertung existiert, liest der Userspace-PEP die gepinnten Policies und Attribute selbst und führt die erforderliche Entscheidungsauswertung lokal aus.

Diese Entscheidung ergänzt die Grenzen des LSM-Hooks um eine nachträgliche Prüfung. Der Kernel bleibt für neue Dateiöffnungen der unmittelbare Durchsetzungspunkt. Der Userspace-PEP behandelt dagegen bestehende Zugriffssituationen, die nach der ursprünglichen Entscheidung weiterlaufen. Damit wird der dynamische Charakter von ASBAC im Prototyp sichtbar, ohne dass jede spätere Dateioperation erneut über denselben LSM-Hook modelliert werden muss.

Eine Alternative wäre, bei einer nachträglichen Policy-Verletzung den gesamten Prozess zu beenden. Diese Variante wäre einfacher, aber grobgranular. Ein Prozess kann mehrere Dateien geöffnet haben, von denen nur eine gegen die aktuelle Policy-Lage verstößt. Das Beenden des gesamten Prozesses würde auch legitime Zugriffe desselben Prozesses abbrechen. Eine andere Alternative wäre, bestehende Zugriffe nicht nachträglich zu behandeln. Dann würde der Prototyp jedoch einen wesentlichen Teil der ASBAC-Fragestellung nicht abdecken.

Die Konsequenz ist, dass neben der kernelbasierten Zugriffskontrolle eine zusätzliche Userspace-Komponente erforderlich ist. Diese Komponente arbeitet beobachtend und periodisch, nicht als Ersatz für den LSM-Hook. Sie erweitert den Prototyp um die Fähigkeit, bestehende Dateizugriffe nach Änderungen der Entscheidungsgrundlage erneut zu bewerten.

#### 4.6.14 Enforcement durch Schließen von File Descriptors

Wenn der Userspace-PEP eine bestehende Policy-Verletzung erkennt, muss der Prototyp den unzulässig gewordenen Zugriff beenden können. Bei Dateizugriffen wird der fortdauernde Zugriff über einen File Descriptor repräsentiert. Dieser File Descriptor ist daher der konkrete Ansatzpunkt, um genau den betroffenen Zugriff zu entziehen.

Die Entscheidung lautet, Enforcement durch das Schließen des betroffenen File Descriptors umzusetzen. Der Userspace-PEP ermittelt Prozess-ID und File-Descriptor-Nummer der verletzenden Zugriffssituation. Anschließend schließt die Enforcement-Komponente



genau diesen File Descriptor im betroffenen Prozess. Im aktuellen Prototyp geschieht dies auf x86\_64-Linux über `ptrace`, indem im Zielprozess ein `close`-Systemaufruf für den ausgewählten File Descriptor ausgeführt wird.

Diese Entscheidung ist feingranularer als ein Prozessabbruch. Wenn ein Prozess mehrere Dateien geöffnet hat, kann nur der File Descriptor geschlossen werden, der zu dem unzulässig gewordenen Zugriff gehört. Andere File Descriptors desselben Prozesses bleiben bestehen, sofern sie nicht selbst gegen eine Policy verstoßen. Damit passt die Maßnahme zu der Anforderung, nur den betroffenen Zugriff zu beenden.

Eine Alternative wäre, den gesamten Prozess zu terminieren. Dies wäre einfacher umzusetzen, aber deutlich invasiver. Eine weitere Alternative wäre, lediglich eine Verletzung zu protokollieren und kein Enforcement vorzunehmen. Diese Alternative wird nicht verwendet, weil eine erkannte Verletzung im Prototyp unmittelbar den Entzug des betroffenen Zugriffs auslösen muss.

Die Konsequenz ist, dass das Enforcement betriebssystem- und architekturabhängig wird. Der aktuelle Ansatz ist auf x86\_64-Linux ausgelegt und benötigt ausreichende Berechtigungen für den Eingriff in fremde Prozesse. Zudem ist das Schließen fremder File Descriptors ein invasiver Eingriff, der sorgfältig auf den tatsächlich betroffenen FD begrenzt werden muss. Für den Prototyp ist diese Einschränkung akzeptabel, weil sie eine gezielte Behandlung bestehender Dateizugriffe ermöglicht.

#### **4.6.15 Administration und Diagnose über ein separates Tool**

Für Betrieb und Evaluation des Prototyps muss sichtbar sein, welche Policies, Generationen und Attribute aktuell im Kernelzustand vorliegen. Diese Informationen liegen nicht nur in den ursprünglichen Textdateien, sondern auch in der übersetzten Map-Repräsentation. Gerade bei dynamischen Attributen und gehashten Attributnamen ist eine Diagnosemöglichkeit wichtig, um nachvollziehen zu können, welche Werte tatsächlich geladen wurden.

Die Entscheidung lautet, Administration und Diagnose über ein separates, lesendes Werkzeug umzusetzen. Das Administrationswerkzeug öffnet die gepinnten Maps und gibt aktive Policy-Bänke, statische Policies, Stream policies und Attribute aus. Für gehashte Attributnamen und Zeichenkettenwerte baut es zusätzlich ein Wörterbuch aus den Policy- und Attributdateien auf. Dadurch können viele Hashwerte wieder einer lesbaren Bezeichnung zugeordnet werden, ohne dass die Kernel-Maps selbst Klartextnamen speichern müssen.

Diese Trennung verhindert, dass Diagnosefunktionen mit der eigentlichen Policyverwaltung vermischt werden. Der Loader bleibt für das Verarbeiten und Schreiben gültiger Zustände zuständig. Das Administrationswerkzeug dient dagegen der Sichtbarkeit des aktuellen Zustands. Es arbeitet bewusst lesend, damit es den Kernelzustand nicht unbeabsichtigt verändert und nicht in Konkurrenz zum Policy-Sync tritt.

Eine Alternative wäre die direkte Nutzung allgemeiner Werkzeuge wie `bpftool`. Damit lassen sich Maps zwar grundsätzlich inspizieren, die Ausgabe ist jedoch stärker an der binären Map-Struktur orientiert und für die fachliche Bewertung weniger komfortabel. Eine weitere Alternative wäre, Diagnose direkt in das Hauptprogramm oder den Userspace-PEP einzubauen. Dies würde jedoch Zuständigkeiten vermischen und die Beobachtung des Zustands stärker an einen laufenden Prozess binden.

Die Konsequenz ist ein zusätzliches Werkzeug im Prototyp. Dafür können gelade-

ne Policies und Attribute unabhängig von der Policyverwaltung eingesehen werden. Die Diagnose bleibt allerdings teilweise best-effort: Wenn ein Hashwert nicht aus den vorhandenen Policy- oder Attributdateien rekonstruiert werden kann, kann das Tool ihn nur als Hash beziehungsweise unbekannten Wert anzeigen.

## 5 Entwurf und Umsetzung

Auf Grundlage der zuvor entwickelten Konzeption beschreibt dieses Kapitel deren technische Ausgestaltung im Prototyp. Zunächst wird die Implementierungsstruktur des Rust-Workspaces dargestellt. Anschließend werden die gemeinsamen Datenstrukturen und die gepinnten eBPF-Maps als Schnittstelle zwischen Userspace und Kernelspace erläutert. Darauf aufbauend werden die Initialisierung des Prototyps, die Verarbeitung von Policies und dynamischen Attributen sowie die Zugriffskontrolle für neue und bereits bestehende Dateizugriffe beschrieben. Den Abschluss bilden die Administrationsfunktion, die Fehlerbehandlung sowie die technischen Grenzen von Build und Zielsystem.

### 5.1 Implementierungsstruktur

#### 5.1.1 Aufteilung des Rust-Workspaces

Die Umsetzung ist in mehrere Rust-Crates mit klar abgegrenzten Verantwortlichkeiten aufgeteilt. Diese Struktur bildet die zuvor begründete Trennung zwischen Userspace und Kernelspace technisch ab. Das Crate `tails-pdp-ebpf` enthält den eBPF-Programmcode, der im Kernel ausgeführt wird und deshalb den dort geltenden Einschränkungen unterliegt. Die Userspace-Crates übernehmen dagegen Initialisierung, Policy- und Attributverarbeitung, Überwachung bestehender Zugriffe sowie Administration. Gemeinsame Datenstrukturen befinden sich in einem separaten Crate, damit Userspace und eBPF-Programm dieselben Map-Layouts verwenden.

| Crate                                   | Aufgabe                                                                                                                                                                      |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>tails-pdp</code>                  | Hauptprogramm zum Laden des eBPF-Objekts, Einrichten der gepinnten Maps und Tail Calls sowie Starten der Userspace-Komponenten.                                              |
| <code>tails-pdp-policy-loader</code>    | Einlesen, Validieren und Übersetzen von Policydateien sowie Aktivieren vollständiger Policy-Generationen.                                                                    |
| <code>tails-pdp-attribute-loader</code> | Bereitstellung und Aktualisierung von Zeit-, System-, Subject- und Ressourcenattributen.                                                                                     |
| <code>tails-pdp-userspace-common</code> | Gemeinsame Hilfsfunktionen und Zugriff auf die gepinnten eBPF-Maps für Userspace-Komponenten.                                                                                |
| <code>tails-pdp-userspace-pep</code>    | Überwachung bestehender Dateizugriffe und gezielter Entzug unzulässiger File Descriptors.                                                                                    |
| <code>tails-pdp-common</code>           | Gemeinsam verwendeter Code mit Datenstrukturen und Auswertungslogik, insbesondere Policy-, Attribut- und Map-Strukturen.                                                     |
| <code>tails-pdp-ebpf</code>             | Enthält die im Kernel ausgeführten eBPF-LSM-Programme für den Hook <code>file_open</code> , die Policy-Auswertung, Tail Calls und die Durchsetzung der Zugriffsentscheidung. |
| <code>tails-pdp-admintool</code>        | Administrationstool zur Anzeige aktiver Policies, Attribut und Generationen aus gepinnten eBPF-Maps.                                                                         |

Tabelle 1: Aufteilung des Rust-Workspaces in Crates und Verantwortlichkeiten

### 5.1.2 Abhängigkeiten zwischen den Crates

Die folgenden Abhängigkeiten beziehen sich auf die im Workspace definierten internen Crates. Zunächst ist zwischen einer regulären Cargo-Abhängigkeit und einer Build-Abhängigkeit zu unterscheiden. Reguläre Abhängigkeiten stehen im Manifestabschnitt `dependencies` und dem Programmcode der jeweiligen Crate zur Verfügung. Build-Abhängigkeiten werden dagegen im Abschnitt `build-dependencies` angegeben und können ausschließlich durch das Build-Skript `build.rs` verwendet werden; sie stehen dem eigentlichen Programmcode nicht automatisch zur Verfügung [19].

Im Prototyp stellt `tails-pdp-ebpf` einen Sonderfall dar. Die Crate ist technisch als Build-Abhängigkeit von `tails-pdp` eingetragen. Das Build-Skript ermittelt ihr Manifest und veranlasst mit `aya-build` die separate Erzeugung des eBPF-Artefakts. Das Artefakt wird anschließend in das Hauptprogramm eingebettet und zur Laufzeit geladen. Es handelt sich daher nicht um eine reguläre Bibliotheksabhängigkeit mit Funktionsaufrufen zwischen `tails-pdp` und `tails-pdp-ebpf`. Die Einordnung als Build-Abhängigkeit ist ein dokumentierter Ersatz für die auf stabilem Cargo nicht verwendbare Artefakt-Abhängigkeit. Diese Umsetzung ergibt sich aus `tails-pdp/Cargo.toml` und `tails-pdp/build.rs`.

Die regulären internen Abhängigkeiten sind wie folgt aufgebaut:

- Das Hauptprogramm `tails-pdp` bindet `tails-pdp-attribute-loader`, `tails-pdp-policy-loader`, `tails-pdp-userspace-common`, `tails-pdp-userspace-pep` und `tails-pdp-common` ein.
- `tails-pdp-attribute-loader`, `tails-pdp-policy-loader` und `tails-pdp-userspace-pep` verwenden jeweils `tails-pdp-common` für gemeinsame Datenstrukturen sowie `tails-pdp-userspace-common` für den Zugriff auf die Userspace-seitigen Map-Funktionen.
- Das Kernelprogramm `tails-pdp-ebpf` hängt ausschließlich von `tails-pdp-common` ab. Dadurch bleiben die gemeinsamen Map-Layouts und Policy-Strukturen zwischen Userspace und Kernelspace konsistent, ohne dass das eBPF-Programm von Userspace-spezifischen Crates abhängt.
- Das eigenständig gestartete `tails-pdp-admintool` verwendet `tails-pdp-common`, liest die gepinnten Maps jedoch selbst und hängt deshalb nicht von `tails-pdp-userspace-common` ab.
- `tails-pdp-common` und `tails-pdp-userspace-common` besitzen keine weiteren internen Crate-Abhängigkeiten. Sie bilden damit die unteren Schichten des Abhängigkeitsgraphen.

Die beiden gemeinsamen Crates bilden unterschiedliche Schnittstellengrenzen. `tails-pdp-common` definiert die ABI-Grenze zwischen eBPF-Programm und Userspace. Map-Schlüssel und -Werte müssen auf beiden Seiten dieselbe Feldreihenfolge, Größe und Ausrichtung aufweisen; die Crate stellt diesen gemeinsamen Datenvertrag bereit. `tails-pdp-userspace-common` kapselt dagegen ausschließlich Userspace-spezifische Funktionen zum Zugriff auf gepinnte Maps. Das eBPF-Programm bleibt von diesen Funktionen unabhängig und verwendet nur die gemeinsamen Datenstrukturen aus `tails-pdp-common`.

Abbildung 2 fasst die strukturellen Abhängigkeiten der projektinternen Crates zusammen. Gleichartige Abhängigkeiten der Userspace-Komponenten werden dabei gemeinsam dargestellt, während die Build-Abhängigkeit des eBPF-Programms gesondert gekennzeichnet ist.

# TODO abhaenigkeiten Diagram,

Abbildung 2: Strukturelle Abhängigkeiten der projektinternen Rust-Crates

## 5.2 Gemeinsame Datenstrukturen und Map-ABI

Die eBPF-Maps bilden die gemeinsame Datenschnittstelle zwischen Userspace und Kernel-space. Sie speichern Schlüssel und Werte als Bytefolgen, denen keine Rust-Typinformationen beigelegt sind. Damit ein im Userspace geschriebener Map-Eintrag vom eBPF-Programm eindeutig gelesen werden kann, müssen beide Ausführungsbereiche dieselbe Binärrepräsentation der verwendeten Datenstrukturen zugrunde legen. Dies betrifft insbesondere Feldreihenfolge, Feldgrößen und Ausrichtung der Schlüssel und Werte.

Diese gemeinsame Repräsentation bildet die Map-ABI des Prototyps. Die hierfür benötigten Strukturen sind in `tails-pdp-common` gebündelt und werden sowohl vom eBPF-Programm als auch von den Userspace-Komponenten verwendet. Die folgenden Unterabschnitte beschreiben die kernelgeeignete Darstellung von Policies und Attributen, die Identifikation von Ressourcen in Map-Schlüsseln sowie Aufbau und Pinning der verwendeten Maps. Das Einlesen, Validieren und Übersetzen der Policy- und Attributdateien wird dagegen in den nachfolgenden Abschnitten behandelt.

### 5.2.1 Kernelgeeignete Policy-Strukturen

Die Policy-Repräsentation ist auf eine konstante Größe und einen vorhersehbaren Kontrollfluss im eBPF-Programm ausgelegt. Für den Hook `file_open` existieren deshalb getrennte Strukturen für statische und attributabhängige Policies. Beide Varianten enthalten die Zugriffsentscheidung, die Aktion, einen Aktivierungsstatus, das Subject sowie die

Bedingungen für Kommando und Ressource. Kommando- und Ressourcenbezeichnungen werden dabei nicht als dynamische Zeichenketten, sondern als Arrays fester Länge abgelegt. Für Ressourcen werden zusätzlich Device- und Inode-Wert gespeichert.

Die Struktur `FileOpenStaticPolicy` enthält nur die Bedingungen, die unmittelbar mit dem Öffnungsvorgang verglichen werden können. `FileOpenStreamPolicy` erweitert diese Repräsentation um eine optionale Zeitbedingung und ein fest dimensioniertes Feld für Attributbedingungen. Jede dieser Bedingungen besteht aus Namespace, Vergleichsoperator, Werttyp, Hash des Attributnamens und dem Vergleichswert. Die Anzahl der aktiven Bedingungen wird gesondert gespeichert; das zugrunde liegende Array besitzt dennoch stets eine feste Obergrenze von vier Einträgen. Damit kann die Auswertung im eBPF-Programm mit begrenzten Schleifen erfolgen, ohne Speicher dynamisch reservieren zu müssen.

Beide Policy-Typen werden als Werte separater Array-Maps gespeichert. Nicht belegte Einträge werden mit einer deaktivierten Policy gefüllt. Die Trennung in statische und attributabhängige Policies erlaubt es, die Auswertung in getrennten Tail-Call-Stufen auszuführen. Die konkrete Aktivierung einer vollständigen Policy-Generation wird in Abschnitt 5.4.5 beschrieben.

### 5.2.2 Repräsentation dynamischer Attribute

Das Attributmodell muss frei wählbare Namen und Werte abbilden, obwohl eBPF-Maps nur Strukturen fester Größe speichern können. Ein Attribut wird deshalb durch einen festen Schlüssel und einen typisierten Wert repräsentiert. `AttributeKey` enthält die aktive Attributbank, den Namespace, zwei Objektidentifikatoren und einen 128-Bit-Hash des Attributnamens. Der Schlüssel legt damit fest, zu welchem System-, Subject- oder Ressourcenobjekt ein Attribut gehört und unter welchem Namen es angesprochen wird.

`AttributeValue` unterscheidet numerische, boolesche und Zeichenkettenwerte. Zahlen und Wahrheitswerte werden direkt als u64 gespeichert. Für Zeichenketten wird nicht der Text selbst, sondern sein 128-Bit-Hash abgelegt. Der Userspace validiert Attributnamen vor der Übernahme und akzeptiert ausschließlich ASCII-Buchstaben, Ziffern, Unterstriche und Bindestriche. Anschließend werden Name und gegebenenfalls Zeichenkettenwert mit derselben Hashfunktion in die feste Kernelrepräsentation übersetzt. Der Kernel muss daher weder Zeichenketten parsen noch dynamischen Speicher verwalten.

Die Policy-Struktur enthält zu jeder Attributbedingung denselben Namenshash sowie den erwarteten Wert. Bei der Auswertung erzeugt das eBPF-Programm aus der aktuellen Attributbank, dem Namespace und der Objektidentität einen `AttributeKey` und liest den zugehörigen Wert aus der Hash-Map. Fehlt ein erforderliches Attribut oder stimmt sein Typ beziehungsweise Vergleichswert nicht überein, gilt die Bedingung als nicht erfüllt. Details zum Einlesen und Validieren der `.attributes`-Dateien behandelt Abschnitt 5.5.

### 5.2.3 Objektidentitäten im Map-Schlüssel

Die beiden Objektidentifikatoren im Attributschlüssel werden abhängig vom Namespace belegt. Für Systemattribute sind beide Werte null, da sie systemweit gelten. Subject-Attribute verwenden die numerische UID als primären Identifikator; der zweite Wert bleibt null. Ressourcenattribute verwenden dagegen Device- und Inode-Wert der Datei.

Diese Zuordnung wird sowohl beim Einlesen der Attributdateien im Userspace als auch bei der eBPF-Auswertung verwendet.

Die Ressourcenidentität wird aus den Dateimetadaten ermittelt, nicht aus dem Pfadnamen. Der Pfad dient beim Attributloader lediglich dazu, die zugehörige Datei zu finden und ihre Metadaten auszulesen. In der Map werden Device und Inode als zwei getrennte u64-Werte gespeichert. Dadurch wird keine verkürzende Hash-Repräsentation der Ressourcenidentität verwendet und Dateien auf unterschiedlichen Dateisystemen bleiben unterscheidbar. Zugleich bleibt die Zuordnung bei Umbenennungen derselben Datei erhalten, solange Device und Inode unverändert bleiben.

Die Objektidentität ist Teil des Map-Schlüssels und damit Teil der ABI. Eine Änderung ihrer Größe, Reihenfolge oder Bedeutung würde den Aufbau der ATTRIBUTES-Map verändern. Bereits gepinnte Maps wären dann nicht mehr kompatibel und müssen vor dem Laden eines Programms mit dem geänderten Layout erkannt werden.

#### 5.2.4 Aufbau und Pinning der eBPF-Maps

Tabelle 2 zeigt die gepinnten Maps, die den dauerhaften Datenaustausch zwischen den Komponenten ermöglichen. Die Policy- und Attributloader schreiben gültige Daten in diese Maps. Das eBPF-Programm liest sie bei der Zugriffsentscheidung; Userspace-PEP und Administrationstool öffnen dieselben Maps bei Bedarf erneut zur Überwachung beziehungsweise Anzeige.

| Map                       | Typ      | Schlüssel und Wert            | Aufgabe                                                                                  |
|---------------------------|----------|-------------------------------|------------------------------------------------------------------------------------------|
| POLICY_GENERATION         | Array    | u32 → u32                     | Speichert die aktive Policy-Generation und bestimmt damit die auszuwertende Policy-Bank. |
| FILE_OPEN_STATIC_POLICIES | Array    | u32 → FileOpenStaticPolicy    | Enthält die statischen file_open-Policies beider Policy-Banken.                          |
| FILE_OPEN_STREAM_POLICIES | Array    | u32 → FileOpenStreamPolicy    | Enthält die attribut- und zeitabhängigen file_open-Policies beider Policy-Banken.        |
| CURRENT_TIME              | Array    | u32 → u64                     | Stellt den aktuellen Unix-Zeitstempel für zeitabhängige Bedingungen bereit.              |
| ATTRIBUTE_GENERATION      | Array    | u32 → u32                     | Speichert die aktive Attributgeneration und damit die aktive Attributbank.               |
| ATTRIBUTES                | Hash-Map | AttributeKey → AttributeValue | Enthält die dynamischen System-, Subject- und Ressourcenattribute.                       |

Tabelle 2: Gepinnte eBPF-Maps als Map-ABI des Prototyps

Beim Start legt tails-pdp das Pin-Verzeichnis /sys/fs/bpf/tails-pdp an und lädt das eBPF-Objekt mit diesem Verzeichnis als Standardpfad für Maps. Gepinnte

Maps bleiben im BPF-Dateisystem erreichbar und können deshalb von später gestarteten Userspace-Komponenten über ihren Pfad geöffnet werden. Die nicht gepinnten Maps für Tail Calls, temporäre Entscheidungen und Debug-Konfiguration gehören dagegen ausschließlich zum geladenen eBPF-Objekt und sind nicht Teil der gemeinsamen Map-ABI.

Da gepinnte Maps einen Programmneustart überdauern können, prüft `tails-pdp` vor dem Laden des eBPF-Programms vorhandene Map-Layouts. Für jede ABI-relevante Map werden Schlüsselgröße, Wertgröße und maximale Eintragszahl mit den aktuellen Strukturdefinitionen verglichen. Bei einer Abweichung wird der Start abgebrochen, statt eine vorhandene Map mit einer möglicherweise falsch interpretierten Byte-Repräsentation weiterzuverwenden. Die Generationswechsel für Policies und Attribute bauen auf diesen stabilen Map-Layouts auf und werden in den jeweiligen Verarbeitungsabschnitten erläutert.

## **5.3 Initialisierung und Laufzeitsteuerung**

### **5.3.1 Laden und Prüfen des eBPF-Programms**

### **5.3.2 Initialisierung der Maps und Tail Calls**

### **5.3.3 Start der Userspace-Komponenten**

## **5.4 Verarbeitung und Laden von Policies**

### **5.4.1 Einlesen der Policydateien**

### **5.4.2 Syntaktische Verarbeitung**

### **5.4.3 Semantische Validierung**

### **5.4.4 Übersetzung in Map-Einträge**

### **5.4.5 Generationenwechsel und Rollback**

## **5.5 Bereitstellung dynamischer Attribute**



- 5.5.1** Aufbau des Attributverzeichnisses
- 5.5.2** Parsen und Validieren der Attribute
- 5.5.3** Attributsschlüssel und Hashwerte
- 5.5.4** Aktualisierung der Attributgeneration
- 5.5.5** Bereitstellung der Zeitinformation
- 5.6 Implementierung des Kernelspace-PEP**
  - 5.6.1** Verarbeitung des file\_open-Hooks
  - 5.6.2** Tail-Call-Kette
  - 5.6.3** Auswertung statischer Policies
  - 5.6.4** Auswertung von Streampolicies
  - 5.6.5** Combining und Durchsetzung
- 5.7 Implementierung des Userspace-PEP**
  - 5.7.1** Ermittlung bestehender Dateizugriffe
  - 5.7.2** Erneute Policy-Auswertung
  - 5.7.3** Entzug des betroffenen File Descriptors
- 5.8 Implementierung des Administrationstools**
- 5.9 Fehlerbehandlung und Konsistenzsicherung**
- 5.10 Build und Deployment auf dem Zielsystem**
  - 5.10.1** Cross-Compilation und Build-Artefakte
  - 5.10.2** Implementierungsbedingte Grenzen

## **6 Evaluation**

### **6.1 Testumgebung**

### **6.2 Testszenarien**

### **6.3 Ergebnisse**

## **7 Zusammenfassung und Ausblick**

## Literatur

- [1] James P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, Air Force Electronic Systems Division, 1972. URL <http://csrc.nist.gov/publications/history/ande72.pdf>.
- [2] eBPF Docs. Pinning. <https://docs.ebpf.io/linux/concepts/pinning/>, 2026. Zugriff am 12.06.2026.
- [3] Antonios Gouglidis, Christos Grompanopoulos, and Anastasia Mavridou. Formal verification of usage control models: A case study of usecon using tla+. *Electronic Proceedings in Theoretical Computer Science*, 272:52–64, 2018. doi: 10.4204/EP TCS.272.5. URL <https://arxiv.org/abs/1806.09848>.
- [4] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. Guide to attribute based access control (abac) definition and considerations. Technical Report NIST Special Publication 800-162, National Institute of Standards and Technology, 2014. URL <https://csrc.nist.gov/pubs/sp/800/162/upd2/final>.
- [5] IOVisor Project. Bcc: Bpf compiler collection. <https://github.com/iovisor/bcc>, 2026. Zugriff am 15.06.2026.
- [6] Michael Kerrisk. bpf(2) linux manual page. <https://man7.org/linux/man-pages/man2/bpf.2.html>, 2026. Zugriff am 12.06.2026.
- [7] Michael Kerrisk. open(2) linux manual page. <https://man7.org/linux/man-pages/man2/open.2.html>, 2026. Zugriff am 12.06.2026.
- [8] Michael Kerrisk. proc\_pid\_fd(5) linux manual page. [https://man7.org/linux/man-pages/man5/proc\\_pid\\_fd.5.html](https://man7.org/linux/man-pages/man5/proc_pid_fd.5.html), 2026. Zugriff am 12.06.2026.
- [9] libbpf Contributors. libbpf. <https://github.com/libbpf/libbpf>, 2026. Zugriff am 15.06.2026.
- [10] OASIS. extensible access control markup language (xacml) version 3.0. <https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-en.html>, 2010. Zugriff am 12.06.2026.
- [11] Rusty Russell. Unreliable guide to hacking the linux kernel. <https://docs.kernel.org/kernel-hacking/hacking.html>, 2026. Zugriff am 12.06.2026.
- [12] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. <https://web.mit.edu/Saltzer/www/publications/protection/>, 1975. Online-Fassung, Zugriff am 12.06.2026.
- [13] The Aya Contributors. Building ebpf programs with aya. <https://aya-rs.dev/book/>, 2026. Zugriff am 12.06.2026.
- [14] The Kernel Development Community. Bpf design q&a. [https://docs.kernel.org/bpf/bpf\\_design\\_QA.html](https://docs.kernel.org/bpf/bpf_design_QA.html), 2026. Zugriff am 12.06.2026.

- [15] The Kernel Development Community. Lsm bpf programs. [https://docs.kernel.org/bpf/prog\\_lsm.html](https://docs.kernel.org/bpf/prog_lsm.html), 2026. Zugriff am 12.06.2026.
- [16] The Kernel Development Community. Bpf maps. <https://docs.kernel.org/bpf/maps.html>, 2026. Zugriff am 12.06.2026.
- [17] The Kernel Development Community. ebpf verifier. <https://docs.kernel.org/bpf/verifier.html>, 2026. Zugriff am 12.06.2026.
- [18] The Kernel Development Community. Linux security module usage. <https://docs.kernel.org/admin-guide/LSM/index.html>, 2026. Zugriff am 12.06.2026.
- [19] The Rust Project Developers. Build scripts, 2026. URL <https://doc.rust-lang.org/cargo/reference/build-scripts.html>.
- [20] Chris Wright, Crispin Cowan, Stephen Smalley, James Morris, and Greg Kroah-Hartman. Linux security modules: General security support for the linux kernel. In *11th USENIX security symposium (USENIX Security 02)*, 2002. URL [https://www.usenix.org/legacy/events/sec02/full\\_papers/wright/wright.pdf](https://www.usenix.org/legacy/events/sec02/full_papers/wright/wright.pdf).